

PROMETEO



Gestor Documental para Gestoría

Desarrollo de Aplicaciones Multiplataforma

Samuel Negrín Méndez

Luz María Álvarez Moreno

PROMETEO

DEDICATORIA

A todas las personas que han hecho posible que pueda dedicarme en cuerpo y alma a este proyecto durante tanto tiempo, en especial a mi familia que me han apoyado en todo momento en esta aventura. También gracias a mis compañeras de la oficina que han sabido mantener todo en orden durante mi ausencia.

ÍNDICES

Contenido

ABSTRACT	5
Resumen y Objetivo del Proyecto	5
Project Summary and Objective	6
JUSTIFICACIÓN DEL PROYECTO	7
INTRODUCCIÓN	8
Principales funciones y problemas que resuelve	8
OBJETIVOS.....	9
Requisitos funcionales del proyecto (RFTP).....	9
RFTP-01 – Autenticación de usuarios	9
RFTP-02 – Gestión de roles de usuario.....	9
RFTP-03 – Gestión de usuarios	9
RFTP-04 – Creación de solicitudes de trámite.....	10
RFTP-05 – Conversión de solicitudes en expedientes.....	10
RFTP-06 – Consulta de expedientes	10
RFTP-07 – Gestión de estados del expediente.....	10
RFTP-08 – Subida y gestión de documentación.....	11
RFTP-09 – Gestión de incidencias.....	11
RFTP-10 – Registro de información del expediente.....	11
DESCRIPCIÓN.....	12
Arquitectura de la aplicación.....	12
Casos de uso	13
Caso de uso: CU01 – Iniciar sesión.....	13
Caso de uso: CU02 – Consultar mis expedientes.....	14
Caso de uso: CU03 – Ver detalle de expediente (CLIENTE).....	15
Caso de uso: CU04 – Subir documentación	16
Caso de uso: CU05 – Crear solicitud de tramite	17
Caso de uso: CU06 – Convertir solicitud en expediente	18
Caso de uso: CU07 – Registrar incidencia.....	19
DISEÑOS	20
Diagrama de clases (UML)	20
Modelo de datos	21
Diagrama E-R (Diseño inicial)	21

Diagrama E-R (Diseño final).....	22
Descripción de las entidades.....	24
Diagrama de flujo de navegación.....	27
Interfaces.....	28
Diseño e implementación de la interfaz de usuario	28
Adaptabilidad y usabilidad	35
TECNOLOGÍA.....	36
IMPLEMENTACIÓN DEL SISTEMA.....	50
Estructura del proyecto	50
Entidades del sistema (Entities).....	51
Repositorios.....	53
Servicios y lógica de negocio.....	53
Controladores MVC.....	54
Gestión de la seguridad	55
Integración con la base de datos	59
Interfaz de usuario.....	59
Despliegue de la aplicación	65
Resultado de la implementación.....	72
METODOLOGÍA DE DESARROLLO.....	73
Planificación inicial del proyecto.....	73
Fases del proyecto.....	74
Pruebas del sistema.....	75
Desviaciones respecto a la planificación inicial	75
Valoración de la metodología utilizada	76
README y GIT	77
Enlace a repositorio GitHub:.....	78
TRABAJOS FUTUROS.....	79
CONCLUSIONES.....	82
REFERENCIAS.....	84
ANEXO 1	86

ABSTRACT

Resumen y Objetivo del Proyecto

El presente proyecto tiene como finalidad el desarrollo de una aplicación web orientada a la gestión documental de expedientes relacionados con la actividad principal de una gestoría especializada en tramitaciones de vehículos. La necesidad de esta aplicación surge de la problemática habitual en este tipo de despachos, donde la documentación se intercambia frecuentemente a través de canales dispersos como correo electrónico o mensajería instantánea, dificultando el control, la organización y el seguimiento adecuado de los trámites.

La plataforma permitirá el acceso tanto a los clientes del despacho como a los empleados de la gestoría mediante un sistema de autenticación con roles diferenciados. De este modo, cada tipo de usuario dispondrá de funcionalidades específicas adaptadas a sus necesidades.

Por un lado, los clientes podrán consultar el estado actualizado de sus expedientes, subir la documentación requerida en formato digital (PDF o imagen), revisar posibles incidencias o correcciones indicadas por la gestoría y mantener un canal de comunicación centralizado dentro de la propia aplicación. Esto facilitará la transparencia del proceso y reducirá errores derivados del envío de documentación incompleta o incorrecta.

Por otro lado, los empleados de la gestoría podrán crear y gestionar expedientes, modificar su estado, revisar la documentación aportada por el cliente y registrar incidencias o solicitudes de información adicional. Cada expediente incorporará información estructurada relevante para el trámite, como datos del vehículo, datos de los interesados, fechas clave y estado actual del procedimiento. Esta estructuración permitirá una mejor organización interna y facilitará la búsqueda y consulta de expedientes tanto para empleados como para clientes.

El objetivo principal del proyecto es desarrollar una aplicación web que centralice y optimice la gestión documental de los expedientes, mejorando la comunicación entre gestoría y cliente, aumentando el control sobre los estados de los trámites y reduciendo la pérdida de información.

Asimismo, este desarrollo sentará las bases para futuras ampliaciones orientadas a la automatización de procesos, como la recepción y clasificación automática de documentación a través de correo electrónico o servicios de mensajería, así como el envío de notificaciones automáticas a los clientes en caso de incidencias o solicitudes pendientes. Estas funcionalidades no formarán parte del alcance inicial (MVP), pero se contemplan como posibles mejoras evolutivas del sistema.

Project Summary and Objective

This project aims to develop a web application focused on the document management of case files related to the main activity of a vehicle processing consultancy firm. The need for this application arises from common issues within this type of business, where documentation is frequently exchanged through scattered channels such as email or instant messaging, making proper control, organization, and follow-up of procedures more difficult.

The platform will allow access to both clients of the firm and employees of the consultancy through an authentication system with differentiated user roles. Each type of user will have access to specific functionalities according to their responsibilities and needs.

On the one hand, clients will be able to check the updated status of their case files, upload required documentation in digital format (PDF or image), review possible incidents or corrections indicated by the consultancy, and maintain a centralized communication channel within the application itself. This will improve transparency in the process and reduce errors caused by incomplete or incorrect document submissions.

On the other hand, consultancy employees will be able to create and manage case files, modify their status, review documentation submitted by clients, and register incidents or requests for additional information. Each case file will include structured information relevant to the procedure, such as vehicle data, applicant details, key dates, and the current status of the process. This structured organization will improve internal workflow management and facilitate the search and consultation of case files for both employees and clients.

The main objective of the project is to develop a web application that centralizes and optimizes document management for case files, enhances communication between the consultancy and its clients, improves control over procedure statuses, and reduces information loss.

Furthermore, this development will lay the groundwork for future expansions aimed at process automation, such as automatic reception and classification of documents via email or messaging services, as well as automated notifications to clients in case of incidents or pending requests. These features will not be part of the initial Minimum Viable Product (MVP), but they are considered potential future improvements of the system.

JUSTIFICACIÓN DEL PROYECTO

Este proyecto nace de una necesidad real basada en mi experiencia personal. Como propietario de una gestoría de vehículos durante más de 10 años, he experimentado toda clase de problemas de comunicación con clientes a la hora de presentar documentación.

A menudo los documentos están muy dispersos tanto en formato como en su ubicación, documentos con firma digital, firma manuscrita, presencial, WhatsApp, email... Prácticamente hay que dedicarle 1 o 2 horas diarias a la clasificación de la documentación.

Otro punto importante también son las consultas de los clientes sobre el estado de sus trámites, si ya está realizado, cuando llega la documentación final, si tiene alguna incidencia o que papel falta... También requiere de una gran dedicación por parte del personal.

Llegados a este punto nos surge la necesidad de crear un producto que nos permita hacer las dos cosas, facilitar la ubicación y gestión de la documentación y, por otro lado, la comunicación en tiempo real con el cliente. Con ello se mejoraría notablemente la productividad de la oficina.

Ya existen gestores documentales en el mercado, actualmente trabajamos con Google Drive, por ejemplo, pero no deja de ser un cajón desastre donde se mete todo sin demasiada organización y la sincronización con los clientes no es la mejor. También utilizamos el servicio de Holded enfocado a los servicios de contabilidad y es un concepto de conexión total con el cliente, pero enfocado a otra actividad diferente.

En relación con el sector del vehículo hay muy pocos productos en el mercado, algunos con restricciones de acceso y mayormente se centran en otro tipo de gestiones, no tanto de almacenamiento y canal de comunicación, sino más bien un software de gestión interna del despacho para lo cual ya tenemos una aplicación de gestión en funcionamiento.

Espero que este proyecto finalmente tenga aplicaciones reales y con las ampliaciones futuras creo que puede resultar una herramienta muy potente de cara al futuro en sector muy competitivo que actualmente se compite principalmente en precio con lo cual es muy importante poder hacer el trabajo de forma eficiente y rápida.

Se contempla también la posibilidad de comercializar este producto al mercado de gestorías a nivel nacional, gracias a las colaboraciones que tenemos con multitud de despachos por todo el territorio nacional y la falta de herramientas en el mercado tan especializadas y personalizables.

INTRODUCCIÓN

Principales funciones y problemas que resuelve

La aplicación desarrollada tiene como objetivo principal centralizar y optimizar la gestión documental de los expedientes tramitados por una gestoría especializada en vehículos. Actualmente, gran parte de la documentación se intercambia mediante canales como correo electrónico o aplicaciones de mensajería, lo que puede generar desorganización, pérdida de información y dificultades en el seguimiento del estado de los trámites.

El sistema propuesto resuelve estos problemas proporcionando una plataforma web unificada en la que clientes y empleados pueden interactuar de forma estructurada y controlada. A través de esta herramienta, se mejora la organización interna, se reduce la posibilidad de errores y se facilita la comunicación entre ambas partes.

Las principales funciones del sistema serán:

Autenticación segura mediante usuario y contraseña.

Diferenciación de roles entre cliente y administrador.

Creación y gestión de expedientes por parte del administrador.

Creación de solicitudes de expedientes por parte del cliente

Consulta de expedientes por parte del cliente.

Visualización y modificación del estado de los expedientes.

Subida de documentación digital asociada a cada expediente.

Registro y gestión de incidencias o solicitudes de documentación adicional.

Consulta de incidencias por parte del cliente.

Almacenamiento de datos estructurados relacionados con el trámite (datos del vehículo, interesados y fechas relevantes).

Estos requisitos definen el alcance funcional del producto mínimo viable (MVP) del proyecto, sobre el cual podrán desarrollarse futuras mejoras orientadas a la automatización y ampliación de funcionalidades.

OBJETIVOS

Requisitos funcionales del proyecto (RFTP)

RFTP-01 – Autenticación de usuarios

Descripción:

El sistema deberá permitir la autenticación de los usuarios mediante un formulario de inicio de sesión basado en email y contraseña.

El acceso a las funcionalidades del sistema estará restringido únicamente a usuarios autenticados.

Prioridad: Alta

RFTP-02 – Gestión de roles de usuario

Descripción:

El sistema deberá gestionar distintos roles de usuario para controlar el acceso a las funcionalidades de la aplicación.

Se establecerán al menos dos roles:

- **Cliente**, que podrá consultar sus solicitudes y expedientes, así como subir documentación.
- **Administrador**, que podrá gestionar solicitudes, expedientes, incidencias y documentación.

Prioridad: Alta

RFTP-03 – Gestión de usuarios

Descripción:

El sistema deberá permitir la creación y gestión de usuarios clientes dentro de la plataforma.

Los usuarios estarán identificados mediante sus datos personales y credenciales de acceso.

La gestión de usuarios será realizada por el administrador del sistema.

Prioridad: Media

RFTP-04 – Creación de solicitudes de trámite

Descripción:

El sistema deberá permitir a los clientes iniciar un trámite creando una solicitud dentro de la plataforma.

Cada solicitud contendrá información básica del trámite solicitado y estará asociada al usuario que la ha creado.

Las solicitudes podrán incluir documentación adjunta aportada por el cliente.

Prioridad: Alta

RFTP-05 – Conversión de solicitudes en expedientes

Descripción:

El sistema deberá permitir al administrador revisar las solicitudes enviadas por los clientes y convertirlas en expedientes de gestión.

Durante este proceso se creará un expediente asociado al cliente y se trasladará la documentación existente en la solicitud al expediente correspondiente.

Prioridad: Alta

RFTP-06 – Consulta de expedientes

Descripción:

El sistema deberá permitir a los clientes consultar los expedientes asociados a su cuenta.

Cada expediente mostrará información relevante del trámite, incluyendo su estado, la documentación asociada y las posibles incidencias registradas.

Prioridad: Alta

RFTP-07 – Gestión de estados del expediente

Descripción:

Cada expediente deberá contar con un estado que permita conocer la situación actual del trámite.

Entre los estados posibles se podrán incluir:

- Pendiente
- En revisión
- Con incidencias
- Finalizado

Estos estados serán gestionados por el administrador.

Prioridad: Media

RFTP-08 – Subida y gestión de documentación

Descripción:

El sistema deberá permitir la subida de documentos asociados a una solicitud o a un expediente.

La subida de documentos podrá ser realizada tanto por clientes como por administradores.

Los archivos se almacenarán en el servidor mientras que sus metadatos se guardarán en la base de datos.

Prioridad: Alta

RFTP-09 – Gestión de incidencias

Descripción:

El sistema deberá permitir al administrador registrar incidencias asociadas a un expediente con el objetivo de solicitar documentación adicional o comunicar problemas detectados durante la tramitación.

Los clientes podrán consultar estas incidencias desde el detalle de su expediente.

Prioridad: Media

RFTP-10 – Registro de información del expediente

Descripción:

Cada expediente deberá almacenar información estructurada relacionada con el trámite, como el tipo de gestión, datos del vehículo, datos del interesado y fechas relevantes del proceso.

Esta información permitirá facilitar la organización y consulta de los expedientes dentro de la plataforma.

Prioridad: Media

DESCRIPCIÓN

Arquitectura de la aplicación

La aplicación se ha diseñado siguiendo una arquitectura web cliente-servidor basada en Spring Boot. Los usuarios acceden a la plataforma a través de un navegador web, que se comunica con el servidor mediante protocolo HTTP/HTTPS.

El servidor está implementado utilizando Spring Boot y se organiza en varias capas: controllers, encargados de gestionar las peticiones HTTP; services, donde se implementa la lógica de negocio; repositories, que gestionan el acceso a la base de datos mediante JPA; y el módulo de seguridad basado en Spring Security, que controla la autenticación y autorización de los usuarios.

La información estructurada de la aplicación se almacena en una base de datos MySQL, mientras que los archivos subidos por los usuarios se guardan en el sistema de almacenamiento del servidor. En la base de datos únicamente se almacenan los metadatos de los documentos, como su nombre, tipo o fecha de subida, manteniendo los archivos físicos en el disco del servidor.

ARQUITECTURA DE LA APLICACION

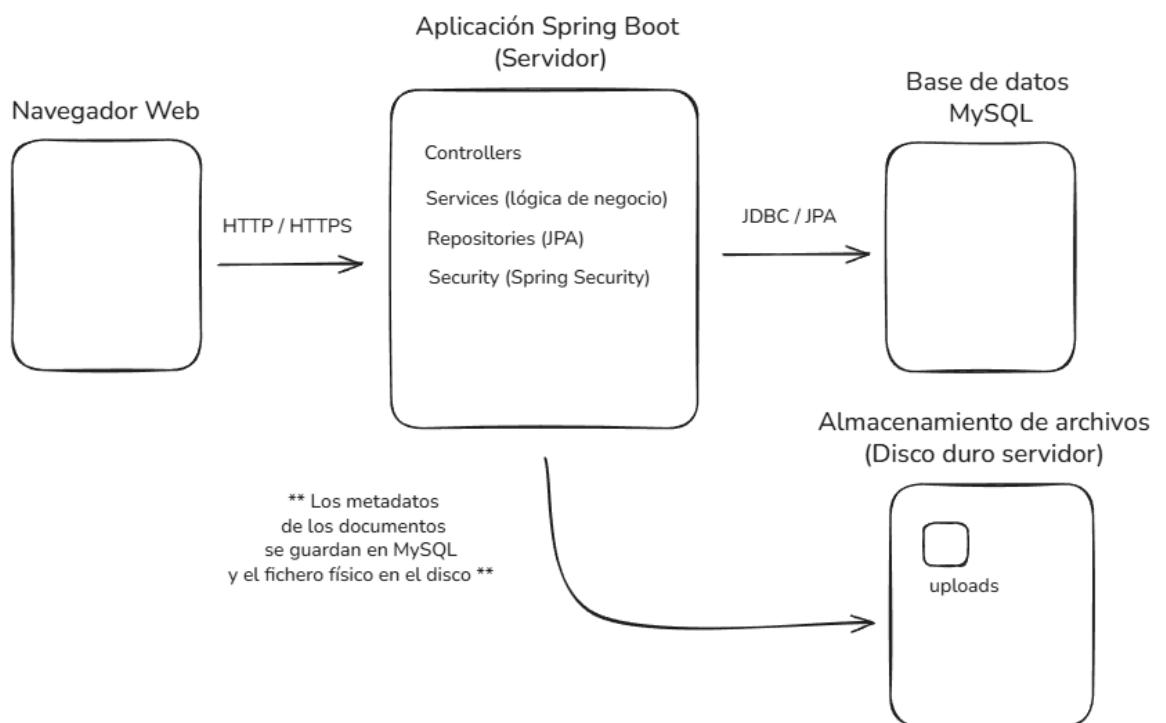


Ilustración 1: Arquitectura de la aplicación

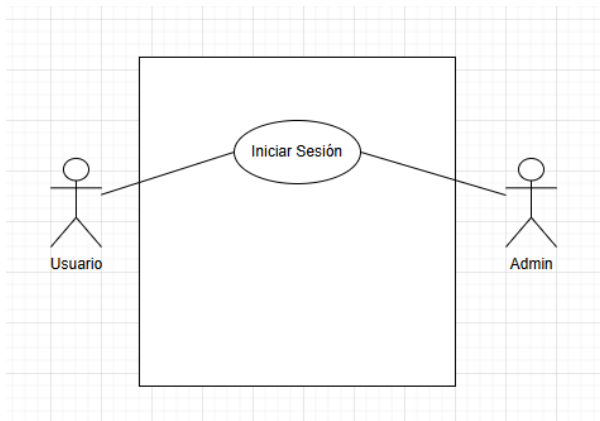
Casos de uso

Caso de uso: CU01 – Iniciar sesión

DESCRIPCIÓN:

Permite al usuario (cliente o administrador) autenticarse en el sistema mediante email y contraseña para acceder a las funcionalidades según su rol.

Ilustración 2: CU01 – Iniciar sesión



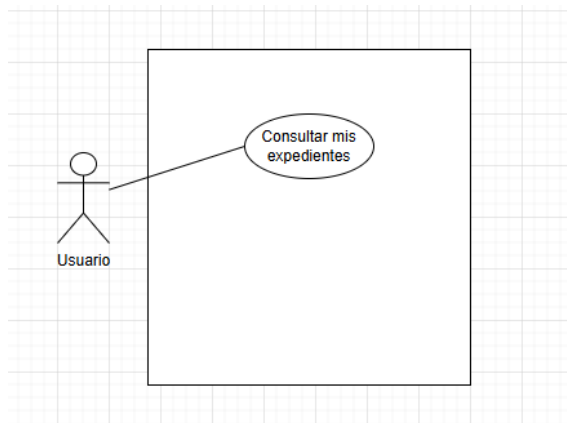
PRECONDICIONES	Usuario registrado Usuario activo
POSTCONDICIONES	Sesión iniciada Acceso al dashboard según rol
DATOS ENTRADA	Email Contraseña
DATOS SALIDA	Usuario autenticado Rol usuario Mensaje error
TABLAS	USUARIO, CLIENTE
CLASES	Usuario UsuarioRepository CustomUserDetailsService SecurityConfig
INTERFACES	login.html cliente_dashboard.html/admin_dashboard.html

Caso de uso: CU02 – Consultar mis expedientes

DESCRIPCIÓN:

Permite al cliente visualizar el listado de expedientes asociados a su cuenta. En el caso de admin puede acceder a un listado con todos los expedientes de todos los clientes.

Ilustración 3: CU02 – Consultar mis expedientes



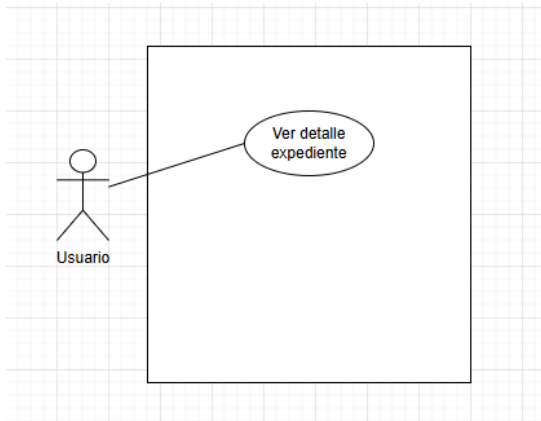
PRECONDICIONES	Usuario logado como CLIENTE
POSTCONDICIONES	Listado de expedientes mostrado
DATOS ENTRADA	IdUsuario (sesión)
DATOS SALIDA	Listado de expedientes
TABLAS	USUARIO CLIENTE EXPEDIENTE
CLASES	Usuario UsuarioService UsuarioRepository Expediente ExpedienteRepository ExpedienteService ExpedienteController
INTERFACES	lista_expedientes.html

Caso de uso: CU03 – Ver detalle de expediente (CLIENTE)

DESCRIPCIÓN:

Permite al cliente consultar la información completa de un expediente propio, incluyendo datos del trámite, documentos asociados, incidencias y todo el historial de cambios.

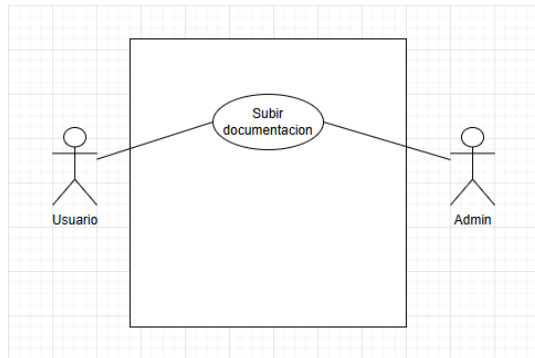
Ilustración 4: CU03 – Ver detalle de expediente



PRECONDICIONES	Usuario logado Expediente pertenece al usuario
POSTCONDICIONES	Detalle del expediente mostrado
DATOS ENTRADA	IdExpediente
DATOS SALIDA	Datos expediente Documentos Incidencias
TABLAS	EXPEDIENTE DOCUMENTO INCIDENCIA HISTORIALCAMBIO
CLASES	Expediente Documento Incidencia ExpedienteRepository DocumentoRepository IncidenciaRepository
INTERFACES	detalle.html

Caso de uso: CU04 – Subir documentación

Ilustración 5: CU04 – Subir documentación



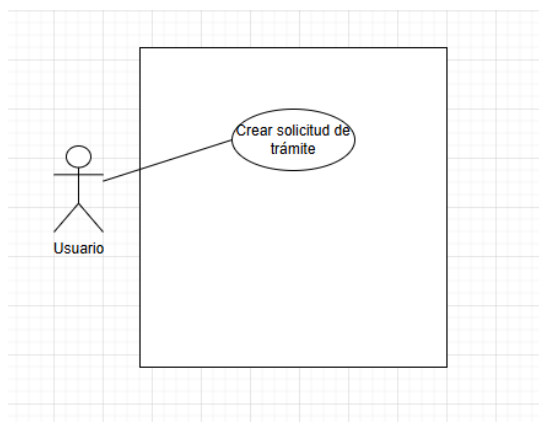
DESCRIPCIÓN:

Permite al usuario subir un archivo asociado a una solicitud o a un expediente. Este caso de uso puede ser ejecutado tanto por el cliente, que aporta documentación requerida para el trámite, como por el administrador, que puede subir documentación generada por la gestoría (justificantes o comprobantes finales del trámite).

PRECONDICIONES	Usuario logado como CLIENTE o ADMIN. Existe una solicitud o expediente válido.
POSTCONDICIONES	Documento almacenado en el servidor. Registro del documento creado en la BD y asociado a solicitud/expediente.
DATOS ENTRADA	IdSolicitud o IdExpediente Archivo
DATOS SALIDA	Confirmación de subida Documento registrado en el sistema
TABLAS	DOCUMENTO USUARIO SOLICITUD EXPEDIENTE
CLASES	Documento DocumentoRepository DocumentoService DocumentoController
INTERFACES	solicitud_detalle.html expediente_detalle.html

Caso de uso: CU05 – Crear solicitud de trámite

Ilustración 6: CU05 – Crear solicitud de trámite



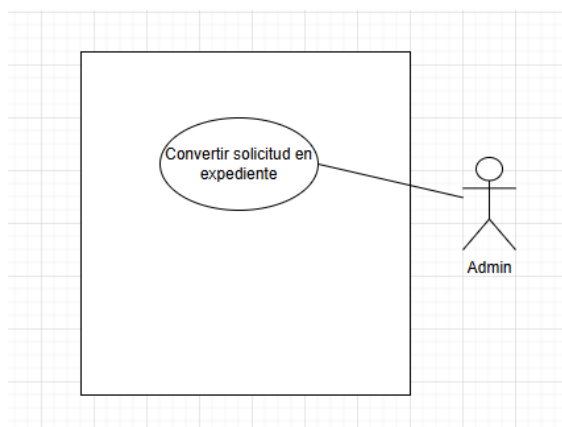
DESCRIPCIÓN:

Permite al cliente iniciar un trámite creando una solicitud con los datos básicos necesarios.

PRECONDICIONES	Usuario logado como CLIENTE
POSTCONDICIONES	Solicitud creada y asociada al cliente
DATOS ENTRADA	Tipo de trámite Datos vehículo Interesados y observaciones (opcional)
DATOS SALIDA	IdSolicitud creada Confirmación
TABLAS	SOLICITUD USUARIO
CLASES	Solicitud SolicitudRepository SolicitudService SolicitudController
INTERFACES	solicitud_form.html solicitudes.html

Caso de uso: CU06 – Convertir solicitud en expediente

Ilustración 7: CU06 – Convertir solicitud en expediente



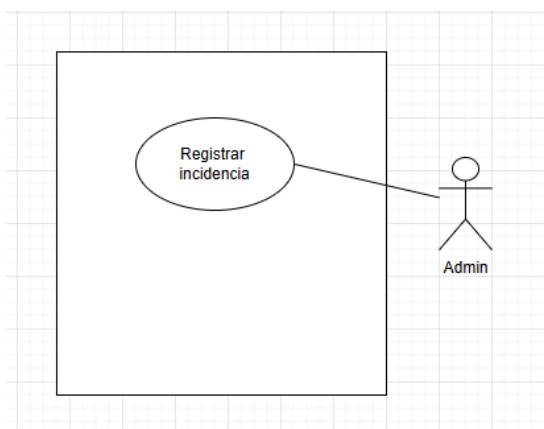
DESCRIPCIÓN:

Permite al administrador validar una solicitud y convertirla en un expediente, trasladando los documentos asociados a la solicitud al nuevo expediente.
Permite rastreo de la solicitud origen para trazabilidad.

PRECONDICIONES	Administrador autenticado Solicitud existente
POSTCONDICIONES	Expediente creado y documentos asociados al expediente
DATOS ENTRADA	IdSolicitud
DATOS SALIDA	IdExpediente creado
TABLAS	SOLICITUD EXPEDIENTE DOCUMENTO
CLASES	Solicitud Expediente Documento SolicitudRepository ExpedienteRepository
INTERFACES	solicitudes.html detalle.html

Caso de uso: CU07 – Registrar incidencia

Ilustración 8: CU07 – Registrar incidencia



DESCRIPCIÓN:

Permite al administrador registrar una incidencia asociada a un expediente/solicitud para solicitar documentación adicional o comunicar un problema al cliente.

PRECONDICIONES	Administrador autenticado Expediente/solicitud existente
POSTCONDICIONES	Incidencia registrada en el sistema
DATOS ENTRADA	IdExpediente / IdSolicitud Tipo Incidencia Descripción
DATOS SALIDA	IdIncidencia creada
TABLAS	INCIDENCIA EXPEDIENTE SOLICITUD
CLASES	Incidencia IncidenciaRepository IncidenciaService IncidenciaController
INTERFACES	detalle.html

DISEÑOS

Diagrama de clases (UML)

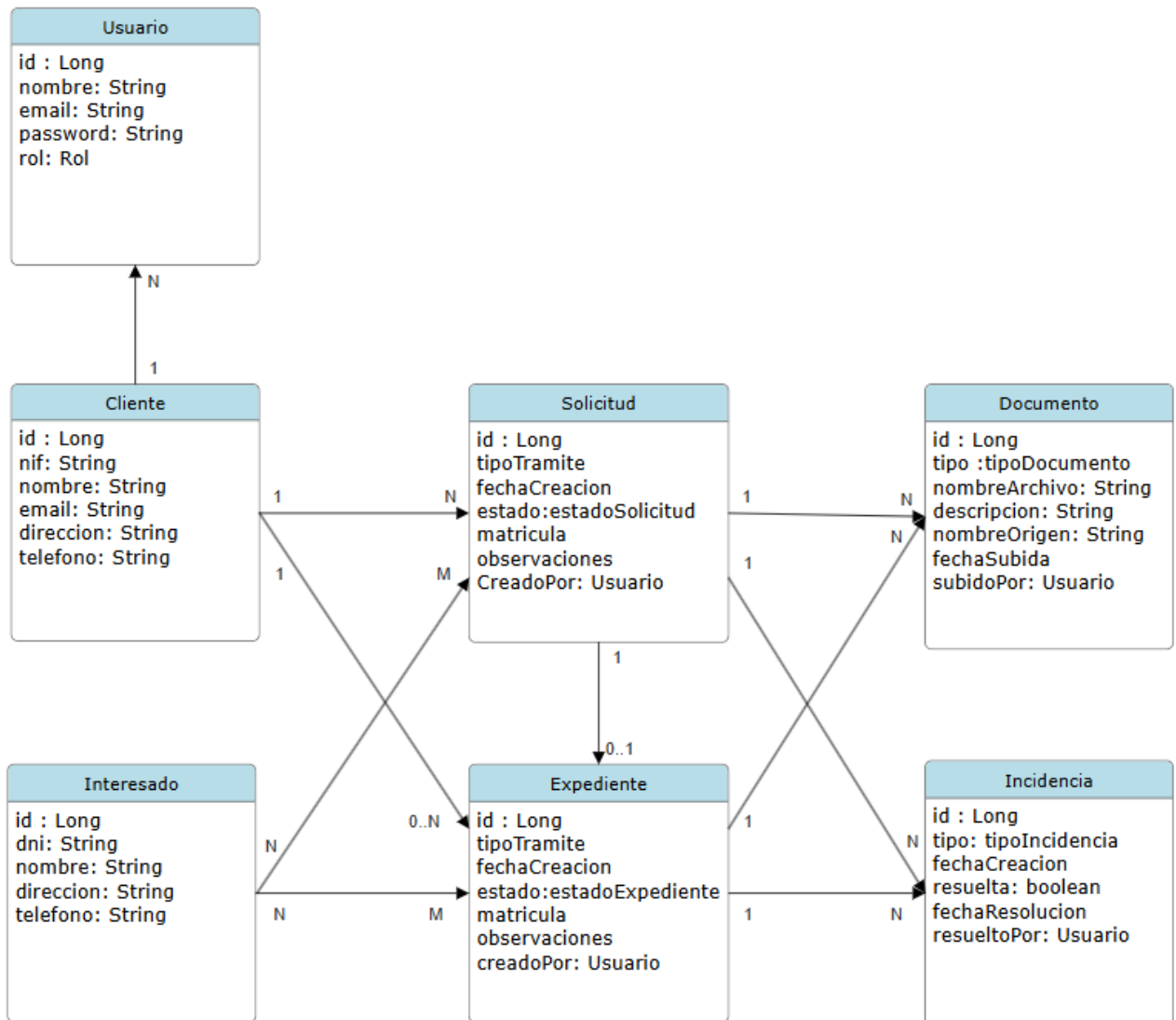


Ilustración 9: Diagrama de clases

Modelo de datos

Para el correcto funcionamiento de la aplicación se ha diseñado un modelo de datos relacional que permite almacenar y gestionar toda la información relacionada con los expedientes y la documentación intercambiada entre la gestoría y sus clientes.

El modelo de datos se ha implementado mediante una base de datos MySQL y se ha estructurado en diferentes entidades que representan los elementos principales del sistema, como clientes, usuarios, solicitudes de trámite, expedientes, documentos e incidencias. Estas entidades se relacionan entre sí mediante claves primarias y claves foráneas que garantizan la integridad referencial de la información almacenada.

El diseño de la base de datos se ha realizado teniendo en cuenta criterios de normalización y escalabilidad, con el objetivo de evitar redundancias y facilitar futuras ampliaciones del sistema. Para ello, algunos elementos como los tipos de trámite o los tipos de incidencia se han modelado mediante tablas independientes que permiten gestionar estos valores de forma flexible.

En el siguiente diagrama se muestra el modelo lógico de la base de datos, donde se representan las tablas que componen el sistema junto con sus campos principales y las relaciones existentes entre ellas.

Diagrama E-R (Diseño inicial)

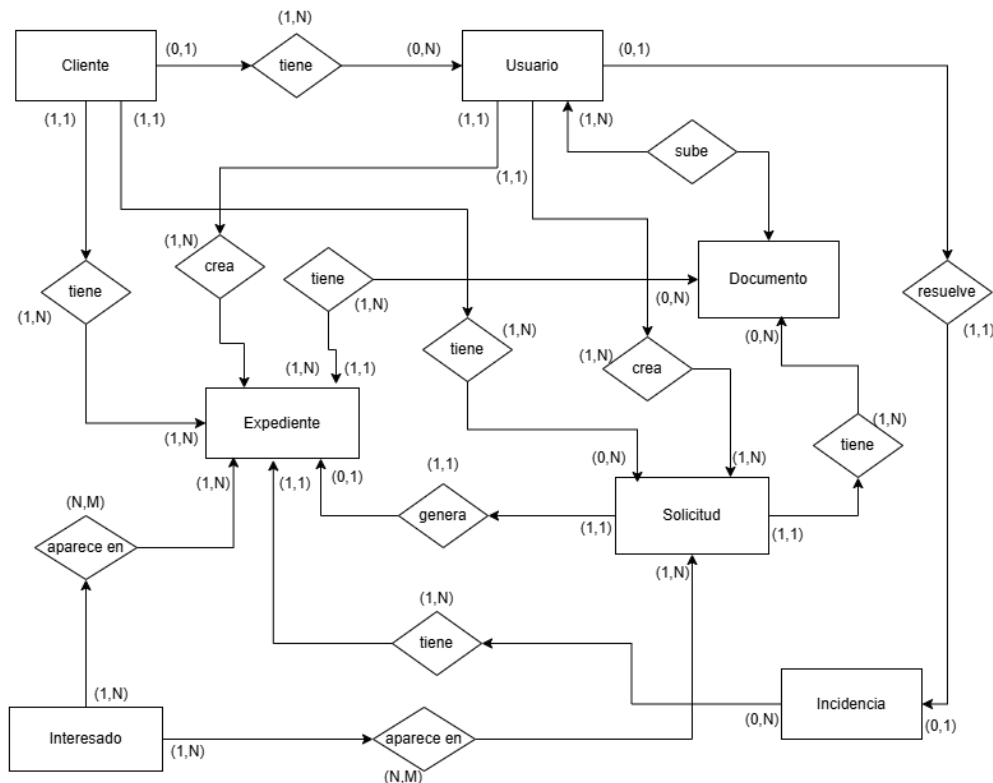
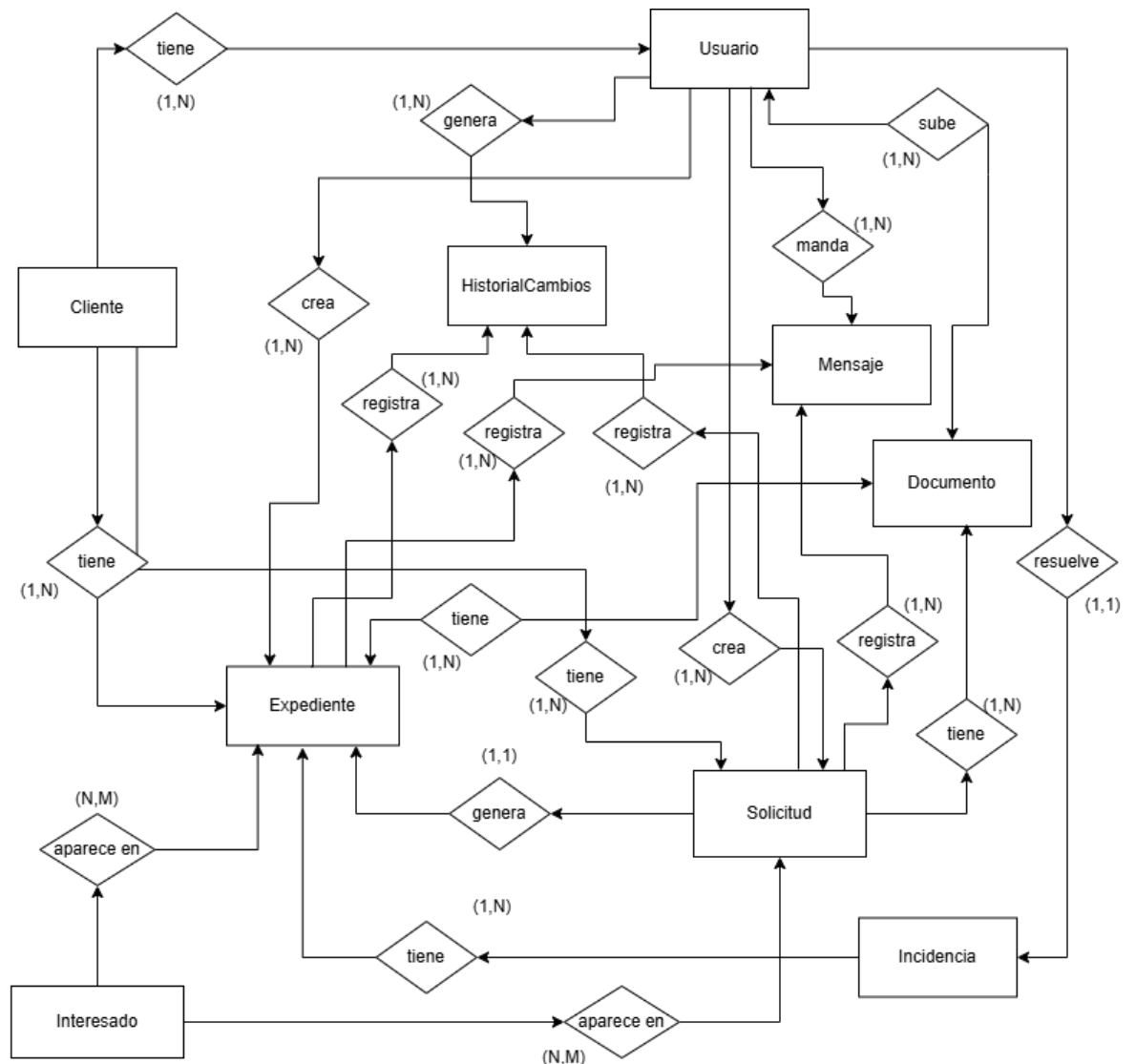


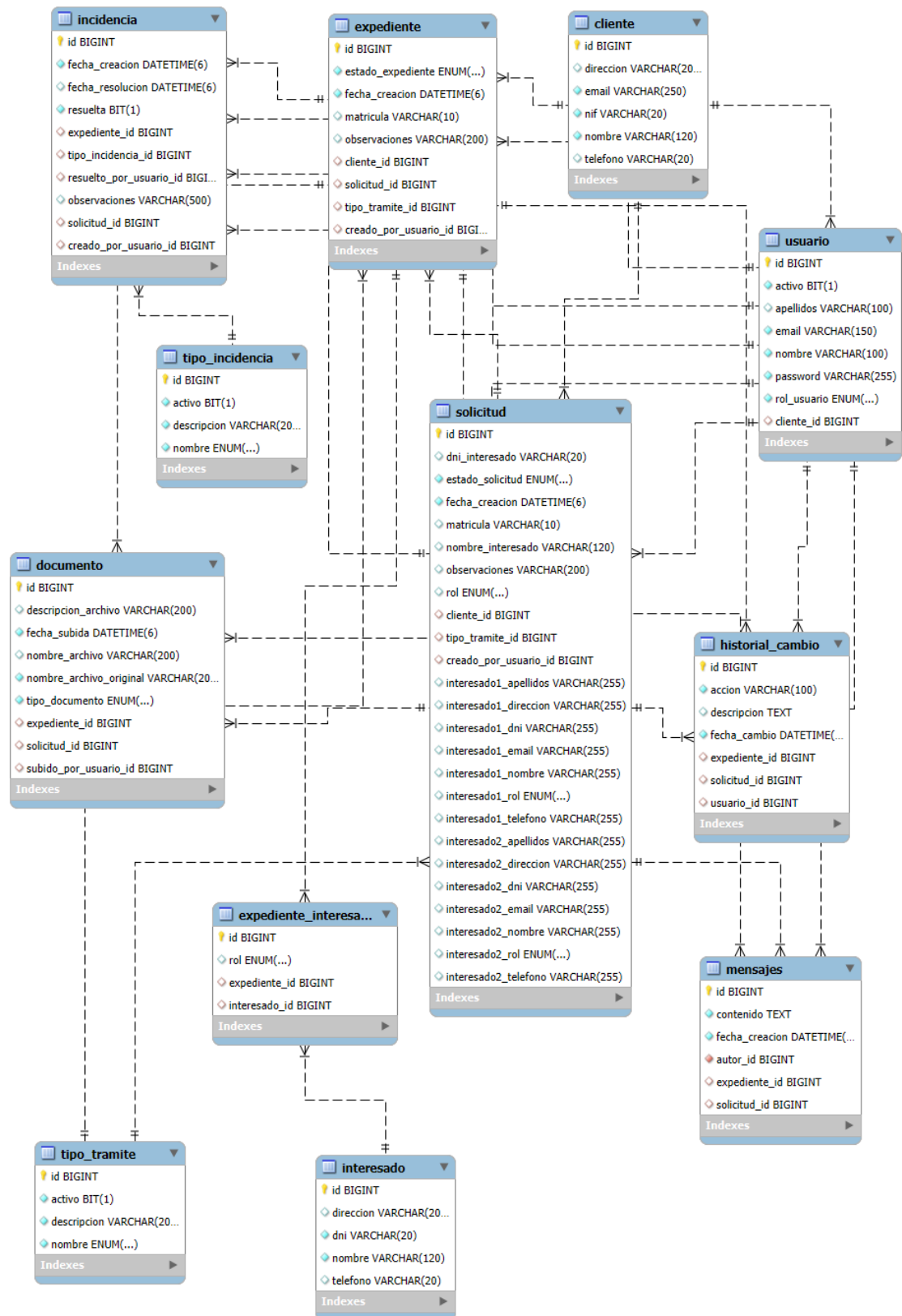
Diagrama E-R (Diseño final)



Durante la fase inicial del diseño se definieron las entidades principales del sistema. Sin embargo, a medida que avanzó el desarrollo y se concretaron mejor los requisitos funcionales, fue necesario ampliar el modelo de datos con nuevas entidades que permitieran dar soporte a funcionalidades adicionales finalmente implementadas. Entre estas ampliaciones destacan las entidades Historial y Mensaje, orientadas al registro de actividad y a la comunicación dentro del sistema.

Estas entidades aportan mucha más trazabilidad y comunicación al sistema permitiendo más información respecto a los expedientes/solicitudes.

Diagrama de la base de datos (Diseño final). Con detalle de campos



Descripción de las entidades

A continuación, se describen las principales entidades que forman parte del modelo de datos de la aplicación.

- **Cliente**

La entidad Cliente almacena la información básica de los clientes de la gestoría que utilizan la plataforma para gestionar sus trámites. En ella se guardan datos como el NIF, nombre, dirección, correo electrónico y teléfono de contacto.

Cada cliente puede disponer de uno o varios usuarios asociados que podrán acceder al sistema mediante credenciales propias.

- **Usuario**

La entidad Usuario representa las cuentas de acceso al sistema. Cada usuario dispone de un nombre, correo electrónico, contraseña cifrada y un rol que determina sus permisos dentro de la aplicación.

Los usuarios pueden pertenecer a un cliente determinado o bien corresponder a usuarios administradores de la gestoría encargados de gestionar los expedientes.

- **Solicitud**

La entidad Solicitud representa una petición inicial de trámite realizada por un cliente. En ella se almacenan datos relevantes como el tipo de trámite solicitado, la fecha de creación, el estado de la solicitud, la matrícula del vehículo implicado y posibles observaciones.

Cada solicitud está asociada a un cliente, a un interesado en el trámite y al usuario que la ha creado.

- **Expediente**

La entidad Expediente representa el trámite una vez que ha sido aceptado y se encuentra en proceso de gestión por parte de la gestoría. El expediente contiene información similar a la solicitud, incluyendo el tipo de trámite, estado del expediente, matrícula del vehículo y observaciones.

Un expediente puede generarse a partir de una solicitud previa y puede contener documentación e incidencias asociadas durante su tramitación.

- **Documento**

La entidad Documento permite almacenar los metadatos de los archivos subidos a la plataforma. Entre la información almacenada se incluye el tipo de documento, el nombre del archivo, su descripción, la fecha de subida y el usuario que lo ha subido.

Los documentos pueden asociarse tanto a solicitudes como a expedientes, permitiendo así el intercambio de documentación entre el cliente y la gestoría durante todo el proceso de tramitación.

- **Incidencia**

La entidad Incidencia se utiliza para registrar posibles problemas o solicitudes de documentación adicional durante la tramitación de un expediente. Cada incidencia incluye el tipo de incidencia, la fecha de creación, su estado de resolución y el usuario que ha gestionado su resolución.

Las incidencias se encuentran asociadas a un expediente concreto.

- **Interesado**

La entidad Interesado representa a las personas implicadas en el trámite, como pueden ser titulares, compradores o vendedores del vehículo. En esta entidad se almacenan datos identificativos y de contacto del interesado.

Un mismo interesado puede aparecer en diferentes expedientes.

- **Expediente_Interesado**

La entidad Expediente_Interesado actúa como tabla intermedia que permite modelar la relación de tipo muchos a muchos entre expedientes e interesados. Esto permite que un expediente pueda tener varios interesados y que una misma persona pueda participar en distintos trámites.

- **Tipo_Tramite**

La entidad Tipo_Tramite almacena los diferentes tipos de trámites que pueden gestionarse en la aplicación, como transferencias de vehículos, duplicados de documentación o matriculaciones. La utilización de una tabla independiente permite ampliar fácilmente los tipos de trámite disponibles sin modificar la estructura de la base de datos.

- **Tipo_Incidencia**

La entidad Tipo_Incidencia recoge las distintas categorías de incidencias que pueden producirse durante la tramitación de un expediente, como la falta de documentación o errores en los datos aportados. Esta tabla permite gestionar de forma flexible los distintos tipos de incidencias que puedan surgir.

Entidades incluidas durante el desarrollo del proyecto:

- **Historial_cambio**

Esta entidad recoge todas las modificaciones realizadas en un expediente, permitiendo saber en todo momento en que momento se realizan los cambios, el usuario que lo ha registrado y la fecha de modificación. De esta manera aparte de tener trazabilidad de eventos y usuarios también permite controlar a nivel operativo los tiempos de respuesta y tramitación.

- **Mensajes**

Esta entidad nos permite registro de mensajes entre usuarios ya sean admin o cliente. Esta entidad nace de la necesidad de implementar un canal de comunicación interno de la aplicación muy importante para explicar posibles cuestiones relacionadas con el trámite dentro de un contexto.

¿Por qué se han añadido estas dos entidades al modelo inicial?

La aplicación tras las primeras etapas del desarrollo veía que cumplía ciertos mínimos de información global del trámite y sus características base, pero estas se presentaban de una forma muy estática, con lo cual perdíamos cierto componente temporal, muy importante en la toma de decisiones y de control del flujo de trabajo.

Por tanto, decidí implementar estas dos entidades para mejorar el detalle de cada expediente de cara a acercar el proyecto a su función principal de favorecer la comunicación y toda la información. Si no controlamos quién hace cada cosa en cada momento es difícil de controlar y tendremos que seguir consultando entre los miembros del despacho. Si no hay un canal de comunicación los clientes si tienen alguna duda seguirán haciendo llamadas al despacho y enviando correos.

Así que, con estos cambios, los usuarios ven todo el historial tanto de comunicación como de cambios en el expediente con lo cual favorece toda la información global. Este es uno de los motivos por los que nace la aplicación, la centralización de la información y la comunicación.

Diagrama de flujo de navegación.

El siguiente diagrama representa el flujo de navegación de la aplicación. A partir de una pantalla de login común, el sistema redirige al usuario a su panel correspondiente según el rol asignado. El rol Cliente permite consultar o crear solicitudes, consultar expedientes y subir documentación. El rol Administrador dispone además de funcionalidades de gestión de expedientes, incidencias y usuarios. Las flechas verdes indican redirecciones tras acciones exitosas.

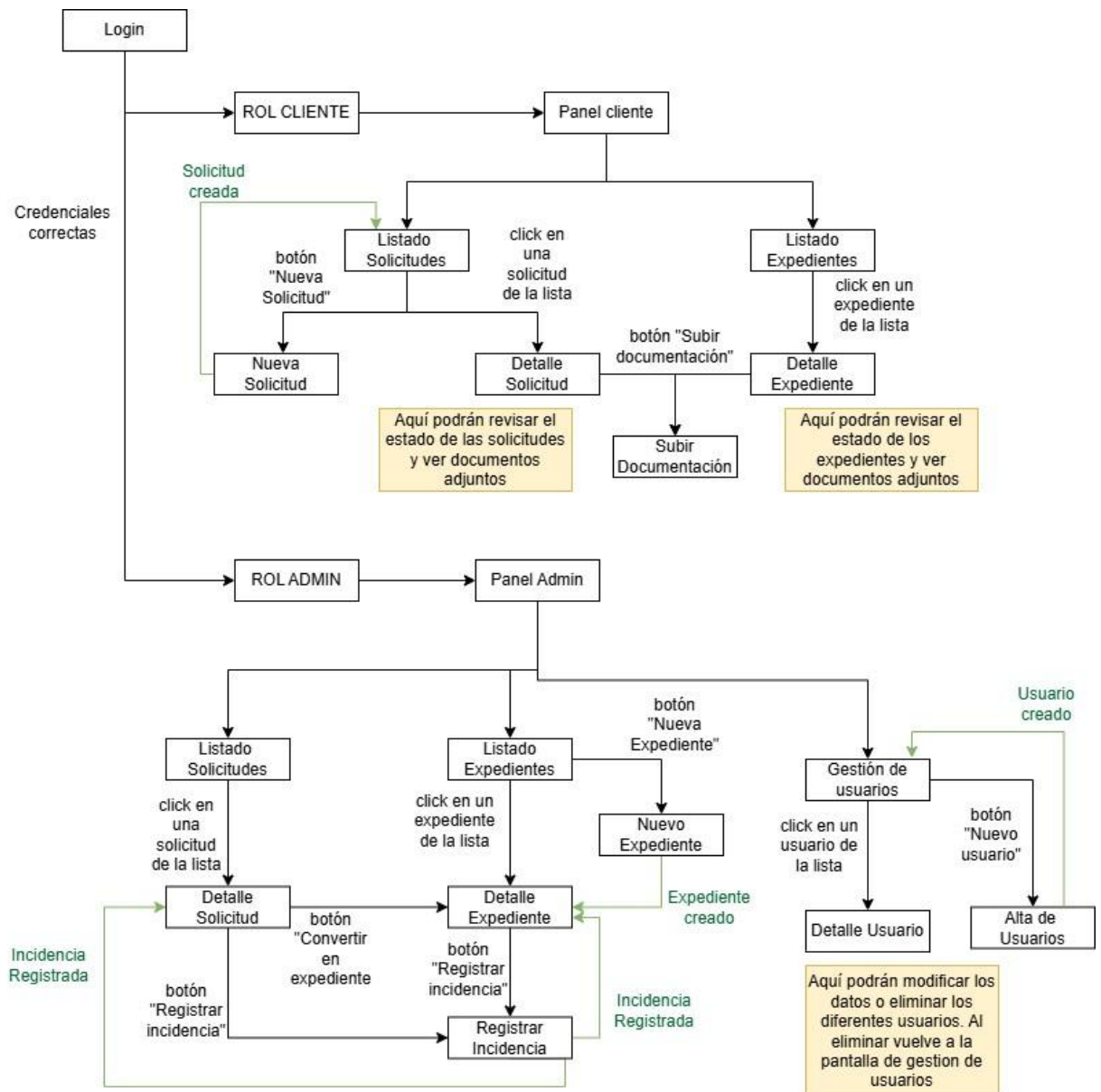


Ilustración 10: Flujo de navegación

Interfaces.

Diseño e implementación de la interfaz de usuario

Introducción

La interfaz de usuario constituye el punto de interacción entre los usuarios y el sistema. Su diseño tiene como objetivo facilitar el acceso a las diferentes funcionalidades de la aplicación de forma clara, intuitiva y eficiente.

En este proyecto se ha optado por una interfaz web accesible desde cualquier navegador, lo que permite a los usuarios acceder al sistema sin necesidad de instalar software adicional. Esta decisión facilita el uso de la aplicación tanto por parte de los clientes de la gestoría como por los empleados que gestionan los expedientes.

La interfaz ha sido desarrollada utilizando **HTML, CSS y el motor de plantillas Thymeleaf**, que permite generar páginas dinámicas integradas con el backend desarrollado en Spring Boot.

El diseño de las diferentes pantallas se ha realizado teniendo en cuenta criterios de simplicidad y facilidad de uso, priorizando una estructura clara de la información y una navegación sencilla entre las distintas secciones de la aplicación.

Antes de comenzar el desarrollo de la interfaz definitiva se realizó una fase de diseño de prototipos o mockups. Estos diseños permiten definir la estructura de las pantallas, la distribución de los elementos y el flujo de navegación entre las diferentes secciones de la aplicación.

El objetivo de esta fase es validar la usabilidad del sistema antes de su implementación, permitiendo detectar posibles mejoras en la organización de la información y en la experiencia de usuario.

A continuación se muestran algunos de los mockups principales que sirven como referencia para el desarrollo de la interfaz final del sistema. Estos mockups se han elaborado en Figma.

Pantalla de inicio de sesión

La primera pantalla a la que acceden los usuarios del sistema es la pantalla de inicio de sesión. En ella se solicita al usuario que introduzca sus credenciales de acceso, compuestas por su dirección de correo electrónico y su contraseña.

Esta pantalla constituye un elemento fundamental del sistema, ya que garantiza que únicamente los usuarios registrados puedan acceder a las funcionalidades de la aplicación.

El proceso de autenticación está gestionado por **Spring Security**, que se encarga de verificar las credenciales introducidas por el usuario y crear una sesión autenticada en caso de que los datos sean correctos.

Una vez validado el acceso, el sistema redirige al usuario a la página principal correspondiente a su rol dentro de la aplicación.

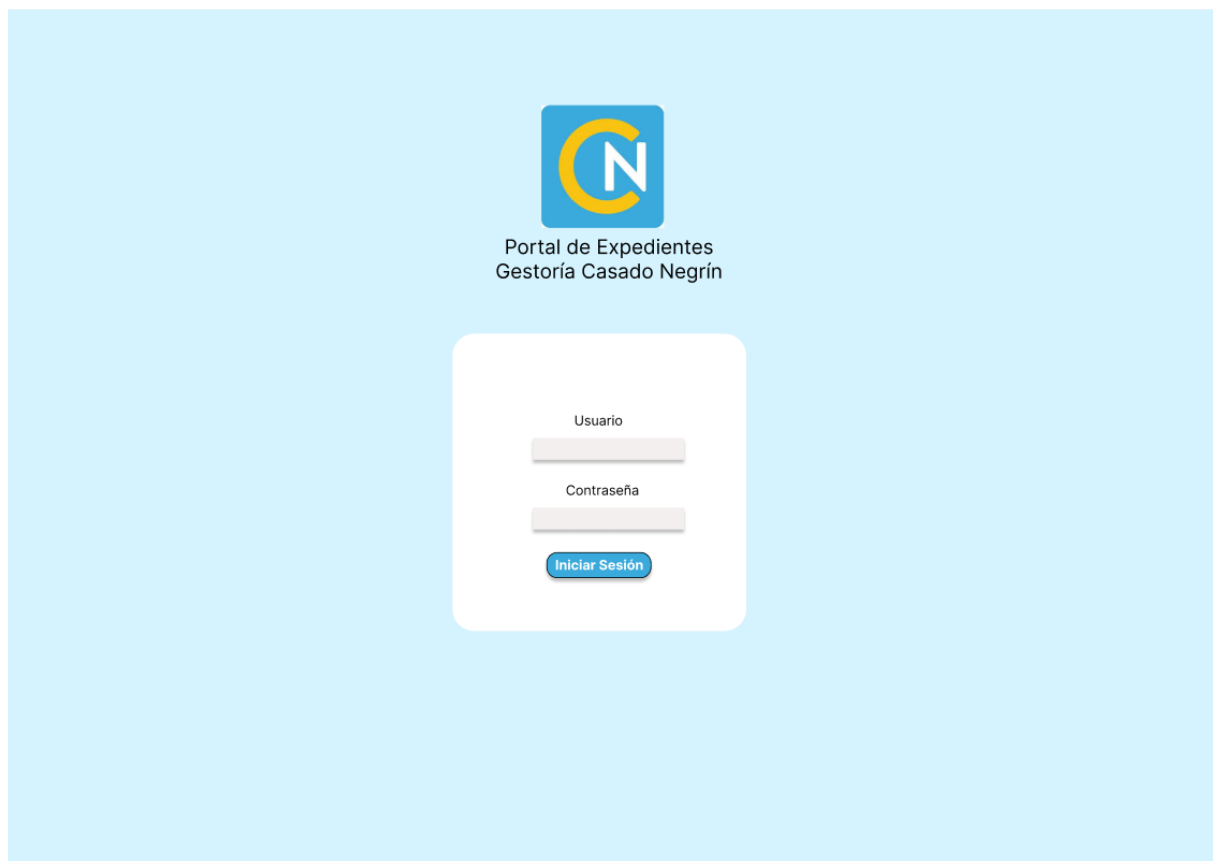


Ilustración 11. Mockup - Login

Panel principal del cliente (Dashboard cliente)

Una vez autenticado, el usuario con rol de cliente accede a su panel principal. Esta sección constituye el punto central desde el que el cliente puede consultar la información relacionada con sus trámites.

Desde este panel el usuario puede acceder principalmente a dos secciones:

- listado de solicitudes realizadas
- listado de expedientes asociados a su cuenta
- creación de nuevo expediente/solicitud

El objetivo de esta pantalla es ofrecer una visión general del estado de los trámites del cliente, permitiéndole acceder rápidamente a la información relevante.

La interfaz se ha diseñado para mostrar la información de forma clara, se le da al cliente una visión general del estado de sus trámites para saber cuántos están tramitándose y en los diferentes estados.

Existirá una versión para cliente y otra para administrador con información similar pero adaptado a cada rol.

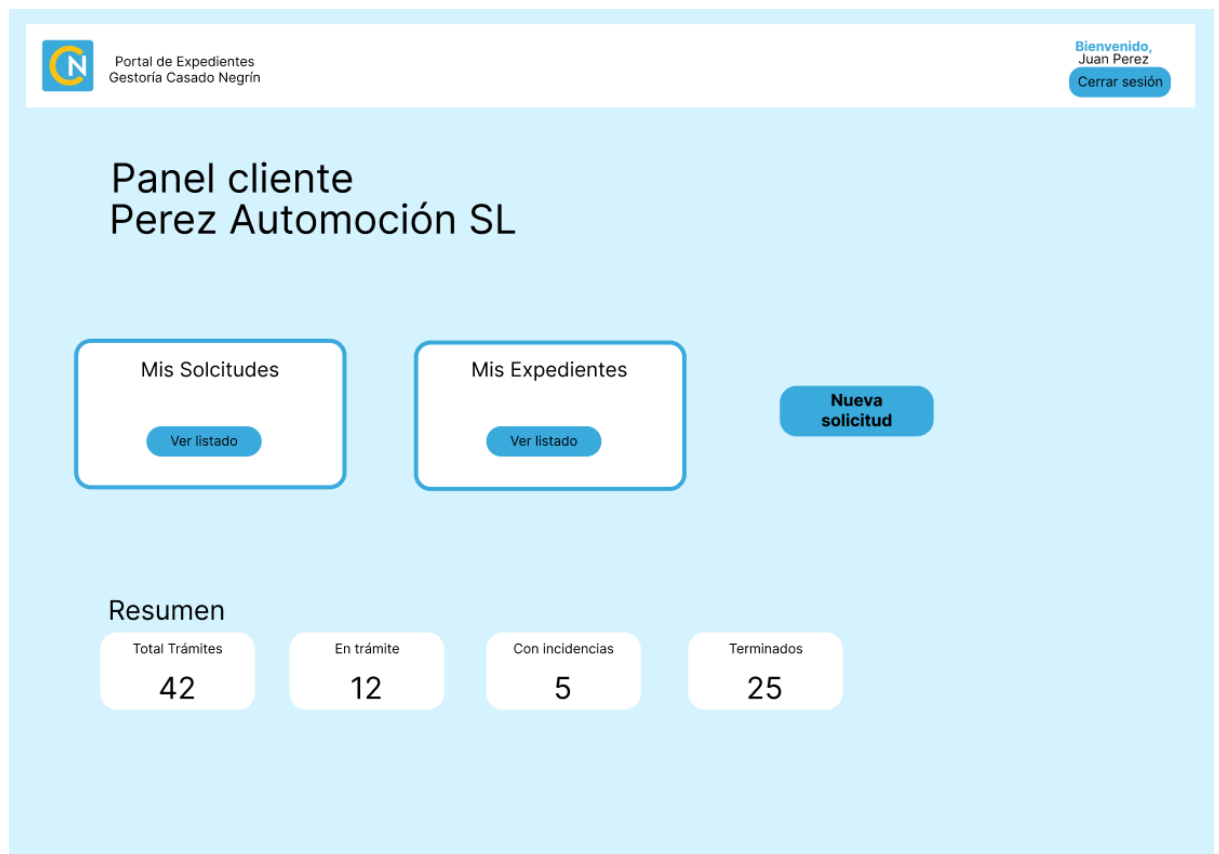


Ilustración 12. Mockup – Panel cliente

Listado de expedientes

La pantalla de listado de expedientes permite al cliente visualizar todos los expedientes asociados a su cuenta.

Cada expediente se muestra como un registro dentro de una tabla o listado que incluye información relevante como:

- tipo de trámite
- matrícula del vehículo
- fecha de creación
- estado del expediente

El usuario puede seleccionar un expediente concreto para acceder a su detalle completo.

Este diseño facilita la consulta rápida de los trámites y permite al cliente conocer en todo momento el estado de su gestión. También tendrá la posibilidad de hacer búsquedas por diversos parámetros y filtrar por estado o tipo de trámite.

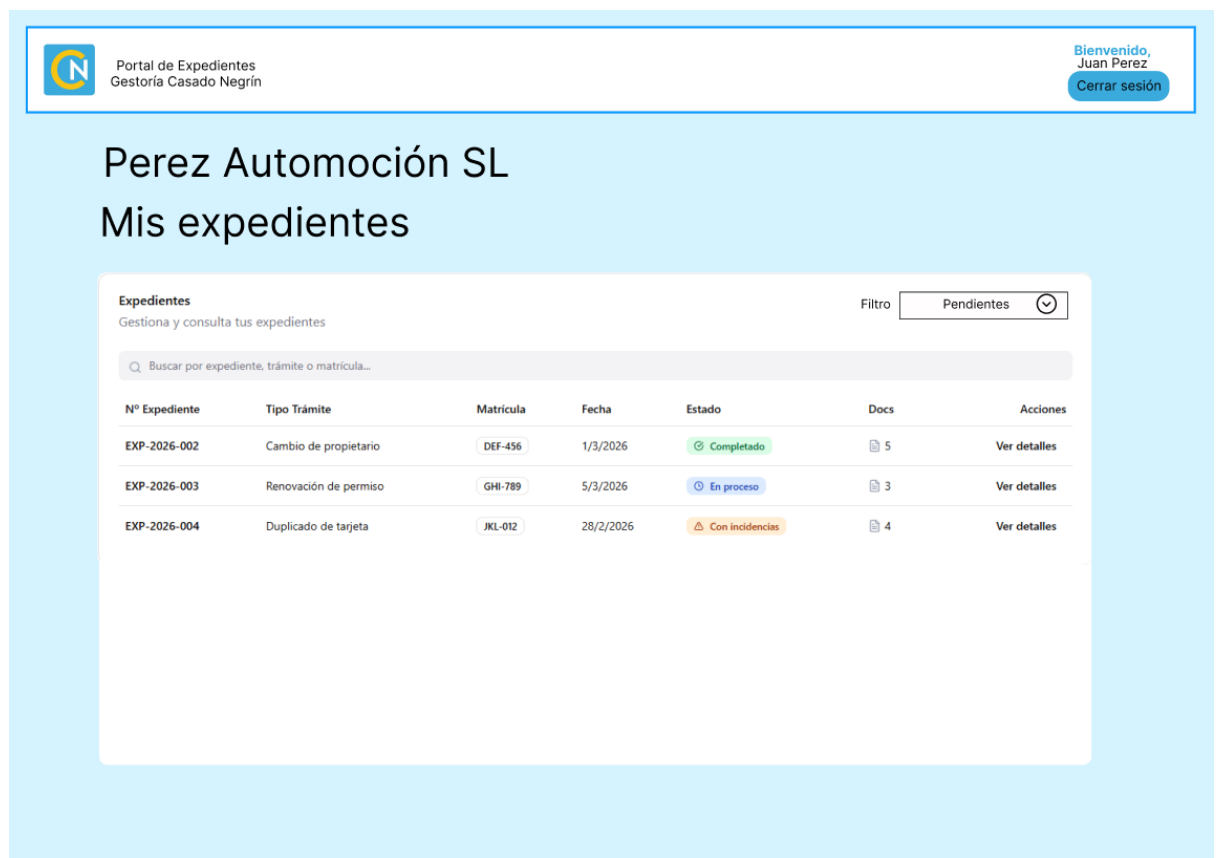


Ilustración 13. Mockup - Lista expedientes (cliente)

Detalle de expediente

La pantalla de detalle de expediente muestra toda la información relacionada con un expediente concreto.

En esta sección se presenta la información estructurada del expediente, incluyendo:

- datos del trámite
- estado actual
- documentos asociados
- incidencias registradas

El objetivo de esta pantalla es centralizar toda la información relevante del expediente en un único lugar, facilitando tanto la consulta por parte del cliente como la gestión por parte del administrador. Esta es la pantalla más importante de la aplicación, representa el núcleo para el que fue creada. Aquí debe estar todo muy bien detallado y mostrar la información de forma atractiva y ordenada, para que el usuario se sienta cómodo a la hora de revisar cada expediente.

Desde esta pantalla el cliente también puede subir nuevos documentos si se requiere información adicional para completar el trámite.

←

Expediente: EXP-2016-001

Datos del trámite

Tipo trámite: Traspaso

Fecha inicio: 01/03/2026

Estado: **En Proceso**

Datos del vehículo

Matrícula: 1234BBB

Fecha matriculación: 01/03/2026

Marca: Opel

Modelo: Corsa

Interesados

Tipo: Comprador

Nombre: Luis Pérez González

DNI: 12345678A

Tipo: Vendedor

Nombre: Jose Hernandez

DNI: 12345678B

Documentos

Documentos del Expediente

Subir Documentación

Nombre	Tipo	Fecha	Subido por	Acciones
DNI Comprador	PDF	2026-03-01	Juan Pérez	
Contrato Compraventa	PDF	2026-03-01	Admin	

Incidencias

No hay incidencias registradas

Ilustración 14. Mockup – Detalle expediente

Creación de solicitud / expediente

Otra de las funcionalidades principales del sistema es la posibilidad de crear expedientes/solicitudes en función del rol del usuario.

Para ello se ha desarrollado un formulario que permite al usuario rellenar información relevante acerca del trámite y adjuntar la documentación necesaria para su tramitación.

El sistema valida el archivo antes de almacenarlo, verificando aspectos como el tipo de archivo o su tamaño.

Una vez creada, para el caso de las solicitudes pasaría a pendiente de revisión por parte de administrador y posteriormente ser convertidas a expedientes. En el caso de expedientes directamente creados por el administrador pasarían directamente a “En trámite”.

Una vez enviados los formularios la vista iría al detalle correspondiente del trámite.

←

Nueva solicitud de trámite

Datos del trámite:

Tipo de trámite

Datos del vehículo:

Matrícula:

Fecha matriculación:

Marca:

Modelo:

Interesados

Comprador

Nombre:

DNI:

Vendedor

Nombre:

DNI:

Documentación (opcional)

Archivo:

Subir Documentación

Crear solicitud

Ilustración 15. Mockup – Nuevo trámite

Panel de administración (Dashboard Admin)

Los usuarios con rol de administrador disponen de un panel de administración que les permite gestionar las diferentes operaciones del sistema.

Desde este panel es posible:

- revisar solicitudes enviadas por los clientes
- crear expedientes a partir de solicitudes
- modificar el estado de los expedientes
- registrar incidencias
- gestionar la documentación asociada a cada trámite

Este panel está diseñado para facilitar el trabajo del personal de la gestoría, permitiendo gestionar los expedientes de forma centralizada.

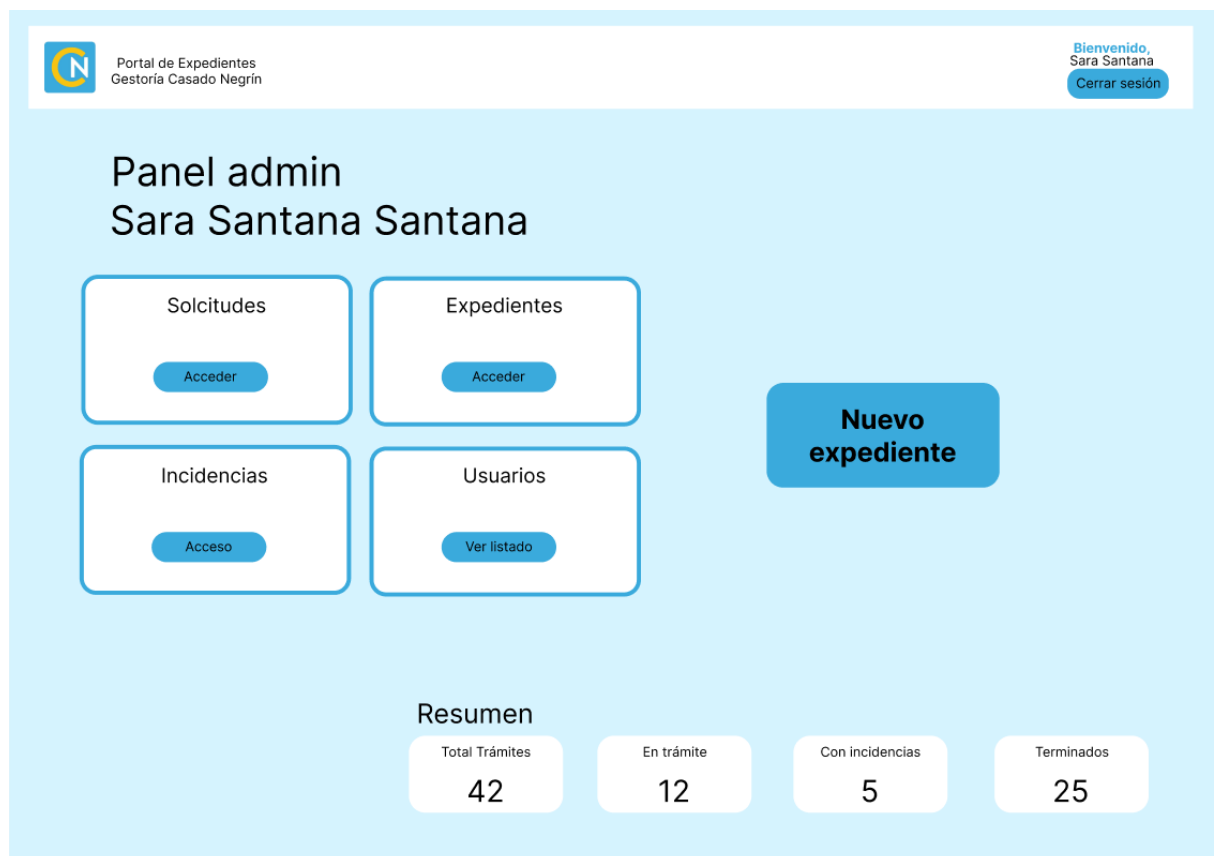


Ilustración 16. Mockup – Panel admin

Adaptabilidad y usabilidad

Durante el diseño de la interfaz se ha tenido en cuenta la importancia de la usabilidad y la claridad visual. La organización de los elementos dentro de cada pantalla sigue una estructura lógica que facilita la navegación entre las diferentes secciones de la aplicación.

Asimismo, se ha buscado mantener una interfaz sencilla y limpia que permita a los usuarios localizar rápidamente la información que necesitan.

Estas plantillas supusieron una base inicial sobre la que construir la interfaz definitiva. Durante la fase de implementación del sistema se rediseñaron y adaptaron los componentes en función de las necesidades que fueron surgiendo durante el desarrollo.

TECNOLOGÍA

Para el desarrollo del sistema de gestión documental se ha utilizado un conjunto de tecnologías orientadas al desarrollo de aplicaciones web empresariales. La elección tecnológica se ha realizado teniendo en cuenta varios criterios: estabilidad, facilidad de mantenimiento, integración entre herramientas, seguridad, escalabilidad y adecuación al tipo de aplicación desarrollada.

La aplicación se ha planteado como una solución web de gestión interna, con acceso diferenciado por perfiles de usuario, almacenamiento persistente en base de datos, subida y consulta de documentos, generación de solicitudes y expedientes, gestión de incidencias y automatización parcial de la lectura de documentos mediante OCR. Por este motivo, se ha optado por una arquitectura basada en Java y Spring Boot, utilizando MySQL como sistema gestor de base de datos y Thymeleaf junto con Bootstrap para la construcción de la interfaz web.

La selección de estas tecnologías permite construir una aplicación robusta, organizada por capas y preparada para futuras ampliaciones sin necesidad de rehacer la estructura principal del proyecto.



Java

El lenguaje principal utilizado para el desarrollo del proyecto ha sido **Java**. Se ha elegido Java porque es uno de los lenguajes más utilizados en el desarrollo de aplicaciones empresariales, especialmente en entornos donde se requiere estabilidad, seguridad, mantenimiento a largo plazo y una buena organización del código.

Java es un lenguaje orientado a objetos, lo que permite representar de forma clara las entidades principales del sistema, como usuarios, clientes, solicitudes, expedientes, documentos, interesados o incidencias. Esta forma de trabajar encaja muy bien con el modelo de negocio del proyecto, ya que cada elemento importante de la aplicación puede transformarse en una clase con sus propios atributos, relaciones y comportamiento.

Uno de los puntos fuertes de Java es su tipado estático. Esto significa que muchos errores pueden detectarse durante la fase de compilación y no únicamente durante la ejecución de la aplicación. En un sistema de gestión documental, donde se manejan datos sensibles, relaciones entre entidades y operaciones sobre expedientes y documentos, esta característica aporta seguridad y fiabilidad.

Además, Java cuenta con un ecosistema muy amplio de librerías, frameworks y herramientas. Esto ha permitido integrar con facilidad tecnologías como Spring

Boot, Spring Security, Spring Data JPA, PDFBox o Tess4J. De esta forma, no ha sido necesario desarrollar desde cero funcionalidades complejas como la autenticación de usuarios, el acceso a base de datos o el procesamiento de documentos PDF.

Frente a otras alternativas como PHP, JavaScript con Node.js o Python con Django, Java se ha elegido por su fuerte orientación al desarrollo empresarial y por su integración natural con Spring Boot. Aunque otras tecnologías también habrían permitido construir la aplicación, Java ofrece una estructura más sólida para proyectos que requieren una clara separación por capas, control de entidades, seguridad y mantenimiento futuro.



Spring Boot

El framework principal utilizado en el proyecto ha sido **Spring Boot**. Esta tecnología permite crear aplicaciones web en Java de una forma más rápida y sencilla que utilizando únicamente Spring Framework tradicional.

Spring Boot proporciona una configuración inicial automática, conocida como autoconfiguración, que reduce considerablemente la cantidad de código de configuración necesario. Esto ha sido especialmente útil en el proyecto, ya que ha permitido centrarse en el desarrollo de las funcionalidades principales sin tener que dedicar demasiado tiempo a configurar manualmente todos los componentes internos de la aplicación.

La aplicación se ha organizado siguiendo una arquitectura por capas, separando controladores, servicios, repositorios, modelos y vistas. Esta separación mejora la claridad del proyecto y facilita su mantenimiento. Por ejemplo, los controladores se encargan de recibir las peticiones del usuario, los servicios contienen la lógica de negocio y los repositorios gestionan el acceso a la base de datos.

Spring Boot también ha facilitado la integración con otras tecnologías utilizadas en el proyecto, como Spring Data JPA, Spring Security, Thymeleaf y MySQL. Esta integración es uno de sus principales puntos fuertes, ya que todo el ecosistema de Spring está diseñado para trabajar de forma conjunta.

Otro motivo importante para elegir Spring Boot es que incluye un servidor embebido, en este caso Tomcat, lo que permite ejecutar la aplicación sin necesidad de instalar y configurar un servidor externo. Esto simplifica tanto el desarrollo como las pruebas, ya que la aplicación puede arrancarse directamente desde el entorno de desarrollo.

Frente a alternativas como Jakarta EE tradicional, Node.js con Express, Laravel o Django, Spring Boot se ha considerado más adecuado para este proyecto por su

equilibrio entre productividad y estructura empresarial. Permite desarrollar rápidamente, pero manteniendo una arquitectura robusta y profesional.



Spring MVC

Dentro de Spring Boot se ha utilizado **Spring MVC** para construir la parte web de la aplicación. Spring MVC sigue el patrón Modelo-Vista-Controlador, que separa la lógica de presentación, la lógica de negocio y los datos.

Este patrón encaja muy bien con el funcionamiento del proyecto. El usuario interactúa con formularios, listados y pantallas de detalle; esas peticiones son recibidas por controladores; los controladores utilizan servicios para procesar la información; y finalmente se devuelve una vista HTML generada con Thymeleaf.

La utilización de Spring MVC permite que el código esté más ordenado y que cada clase tenga una responsabilidad concreta. Por ejemplo, un controlador de expedientes no debería encargarse directamente de guardar documentos o acceder a la base de datos, sino delegar esas operaciones en los servicios correspondientes.

Uno de los motivos por los que se ha elegido Spring MVC frente a una arquitectura completamente separada con API REST y frontend independiente es la simplicidad. Para este proyecto, una aplicación web renderizada desde servidor resulta suficiente y más adecuada, ya que permite desarrollar la interfaz y la lógica de negocio dentro del mismo proyecto, reduciendo la complejidad técnica.

Una alternativa habría sido desarrollar el backend con Spring Boot y el frontend con React, Angular o Vue. Sin embargo, esta opción habría obligado a crear una API REST completa, gestionar autenticación entre frontend y backend, configurar comunicación entre aplicaciones y mantener dos proyectos separados. Para una aplicación de gestión documental interna, Spring MVC con Thymeleaf ofrece una solución más sencilla, coherente y fácil de mantener.



Spring Data JPA

Para gestionar el acceso a la base de datos se ha utilizado **Spring Data JPA**, una tecnología que forma parte del ecosistema Spring y que facilita la implementación de la capa de persistencia.

JPA (Java Persistence API) es una especificación que permite mapear objetos Java a tablas de una base de datos relacional mediante un proceso conocido como **mapeo objeto-relacional (ORM)**.

Mediante este mecanismo, las entidades del sistema se representan como clases Java, cuyos atributos se corresponden con las columnas de las tablas de la base de datos. De esta forma, el desarrollador puede trabajar con objetos Java en lugar de escribir directamente consultas SQL complejas.

Spring Data JPA simplifica aún más este proceso mediante el uso de **repositorios**, que permiten realizar operaciones CRUD (crear, leer, actualizar y eliminar) sobre las entidades sin necesidad de escribir código SQL explícito. Esto permite reducir significativamente la cantidad de código necesario para gestionar la base de datos y facilita el mantenimiento del sistema.

El principal motivo para utilizar Spring Data JPA frente a JDBC tradicional es la reducción de código repetitivo. Con JDBC habría sido necesario escribir manualmente conexiones, consultas SQL, parámetros, conversiones de resultados y control de errores. En cambio, JPA permite trabajar a un nivel más cercano al modelo de objetos de la aplicación.

Frente a alternativas como MyBatis, que ofrece mayor control sobre las consultas SQL, Spring Data JPA se ha considerado más adecuado porque el proyecto se basa en relaciones entre entidades y operaciones CRUD habituales. Además, su integración con Spring Boot es muy directa y facilita el desarrollo de aplicaciones con una estructura limpia y mantenible.



Como implementación de JPA se utiliza

Hibernate, una de las herramientas ORM más extendidas en el ecosistema Java. Un ORM permite mapear clases Java con tablas de una base de datos relacional, facilitando la conversión entre objetos y registros.

Hibernate permite representar relaciones entre entidades de forma natural dentro del código. Por ejemplo, un cliente puede estar relacionado con varios usuarios, una solicitud puede estar asociada a un cliente y a un tipo de trámite, un expediente puede tener documentos asociados, y un expediente puede relacionarse con varios interesados mediante una tabla intermedia.

Esta forma de trabajar es especialmente útil en el proyecto porque el modelo de datos tiene bastantes relaciones. Usar Hibernate evita tener que construir manualmente todas las consultas necesarias para recuperar y asociar información entre tablas.

Otro punto fuerte de Hibernate es que permite mantener una mayor coherencia entre el modelo de clases y el modelo de base de datos. Aunque la base de datos sigue siendo relacional, el código Java puede trabajar con objetos y relaciones, lo que resulta más cómodo y legible.

Frente a trabajar directamente con SQL, Hibernate reduce la cantidad de código técnico necesario y permite centrarse más en la lógica del negocio. No obstante, también requiere diseñar correctamente las relaciones entre entidades para evitar problemas de rendimiento o cargas innecesarias de datos.



Spring Security

La seguridad de la aplicación se ha implementado mediante **Spring Security**. Esta tecnología permite gestionar la autenticación, la autorización y el control de acceso a las distintas rutas del sistema.

En una aplicación de gestión documental, la seguridad es un aspecto especialmente importante, ya que se trabaja con información de clientes, documentos administrativos, expedientes y datos personales. No todos los usuarios deben poder acceder a toda la información, por lo que es necesario establecer un sistema de roles y permisos.

Spring Security permite proteger las rutas de la aplicación y controlar qué usuarios pueden acceder a cada sección. En el proyecto se utiliza para diferenciar el acceso entre usuarios administradores y usuarios clientes. De esta forma, un administrador puede gestionar expedientes, solicitudes o incidencias, mientras que un cliente solo puede consultar o crear información relacionada con su propio ámbito.

Otro punto importante es la gestión del inicio de sesión. Spring Security proporciona mecanismos ya preparados para autenticar usuarios, gestionar sesiones, cerrar sesión y redirigir al usuario según su rol o permisos.

Frente a desarrollar un sistema de seguridad manual, Spring Security ofrece una solución mucho más robusta y probada. Implementar manualmente autenticación, control de sesiones, cifrado de contraseñas y autorización aumentaría el riesgo de errores y vulnerabilidades. Por eso, utilizar una herramienta especializada resulta mucho más adecuado.



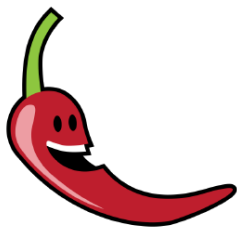
Maven

Para la gestión del proyecto y sus dependencias se ha utilizado **Maven**. Maven permite definir en un único archivo, el pom.xml, las librerías necesarias para que el proyecto funcione.

Gracias a Maven, el proyecto puede descargar automáticamente dependencias como Spring Boot, Spring Data JPA, Spring Security, Thymeleaf, MySQL Connector, PDFBox, Tess4J o Lombok. Esto evita tener que descargar archivos manualmente y facilita que el proyecto pueda ejecutarse en distintos equipos.

Maven también permite gestionar el ciclo de vida de la aplicación: compilar el proyecto, ejecutar pruebas, empaquetar la aplicación y generar el archivo final ejecutable. Esto aporta orden y facilita el trabajo con el proyecto a largo plazo.

Frente a gestionar las dependencias manualmente, Maven reduce errores y asegura que el proyecto tenga siempre las versiones necesarias declaradas de forma clara. Frente a Gradle, que también es una alternativa muy utilizada, Maven se ha elegido por su sencillez, su estructura estándar y su uso habitual en proyectos académicos y empresariales Java.



Lombok

En el proyecto se ha utilizado **Lombok** para reducir código repetitivo dentro de las clases Java. En aplicaciones basadas en entidades, DTOs y modelos, es habitual tener que escribir métodos getter, setter, constructores o métodos auxiliares.

Lombok permite generar parte de este código automáticamente mediante anotaciones, haciendo que las clases sean más limpias y fáciles de leer. Esto resulta especialmente útil en un proyecto con varias entidades y objetos de transferencia de datos.

Su principal ventaja es que mejora la legibilidad del código y reduce la cantidad de líneas necesarias para representar las clases del modelo. De esta forma, el código se centra más en los atributos y relaciones importantes y menos en métodos repetitivos.

Frente a escribir manualmente todos los getters, setters y constructores, Lombok ahorra tiempo y reduce errores. No obstante, debe utilizarse de forma controlada, ya que el código generado no aparece directamente escrito en la clase y puede dificultar la comprensión a personas que no conozcan la herramienta.

**PDFBox[®]****Apache PDFBox**

Para el tratamiento de documentos PDF se ha utilizado **Apache PDFBox**. Esta librería permite trabajar con archivos PDF desde Java, extrayendo información, procesando páginas o dividiendo documentos.

En el contexto del proyecto, PDFBox resulta útil porque la aplicación no solo almacena documentos, sino que también incorpora funcionalidades relacionadas con la lectura y clasificación de documentos administrativos. Al tratarse de un gestor documental, muchos de los archivos manejados por el sistema están en formato PDF.

PDFBox permite manipular este tipo de archivos sin depender de herramientas externas complejas. Esto facilita la integración con el backend de la aplicación y permite que el procesamiento documental forme parte del propio flujo del sistema.

Frente a otras opciones, como iText, PDFBox tiene la ventaja de ser una herramienta ampliamente utilizada y adecuada para tareas de lectura y manipulación de PDFs. Además, resulta suficiente para las necesidades del proyecto, especialmente para dividir documentos o preparar páginas para su posterior análisis mediante OCR.

Tesseract OCR

Para la funcionalidad de reconocimiento óptico de caracteres se ha utilizado el motor OCR Tesseract.

El OCR es una funcionalidad especialmente relevante en el proyecto, ya que permite analizar documentos escaneados o PDFs formados por imágenes para intentar extraer texto de ellos. Esto facilita la detección de tipos de documentos y ayuda a automatizar parte del trabajo de clasificación documental.

Tesseract es un motor OCR ampliamente conocido y utilizado. Tess4J actúa como una capa de integración que permite utilizarlo desde Java. Gracias a esto, el proyecto puede enviar imágenes o páginas convertidas al motor OCR y recibir como resultado el texto detectado.

Se ha elegido esta solución porque permite incorporar OCR de forma local, sin necesidad de enviar documentos a servicios externos. Esto es importante en una

aplicación que puede manejar documentación sensible, ya que evita depender de plataformas de terceros para procesar archivos.

Frente a alternativas basadas en servicios en la nube, como Google Vision, AWS Textract o Azure OCR, Tesseract tiene la ventaja de poder ejecutarse en local y no generar costes por uso. Aunque los servicios en la nube pueden ofrecer mejores resultados en algunos escenarios, también implican dependencia externa, configuración adicional, posibles costes y consideraciones de privacidad. Para el alcance del proyecto, Tesseract mediante Tess4J ofrece una solución adecuada y suficientemente potente.



Thymeleaf

Para la generación de vistas HTML se ha utilizado **Thymeleaf**. Thymeleaf es un motor de plantillas que permite crear páginas dinámicas desde el servidor integrándose directamente con Spring Boot.

En el proyecto, Thymeleaf se utiliza para mostrar formularios, listados, paneles de administración, vistas de detalle, menús laterales, mensajes de confirmación y contenido personalizado según el usuario autenticado.

Uno de los motivos principales para elegir Thymeleaf es que permite trabajar con archivos HTML muy cercanos al HTML estándar. Esto facilita la comprensión de las vistas, ya que el código sigue siendo legible incluso antes de ser procesado por el servidor.

Thymeleaf permite insertar valores dinámicos, recorrer listas, mostrar u ocultar bloques condicionalmente y enlazar formularios con objetos del modelo. Esto resulta fundamental en una aplicación como esta, donde se muestran datos procedentes de la base de datos y se envían formularios para crear solicitudes, expedientes, documentos o incidencias.

Frente a alternativas como JSP, Thymeleaf resulta más moderno, más limpio y mejor integrado con Spring Boot. JSP es una tecnología más antigua y menos recomendable para nuevos desarrollos.

Frente a frameworks frontend como React, Angular o Vue, Thymeleaf tiene la ventaja de ser más sencillo para una aplicación renderizada desde servidor. No requiere construir una API separada ni gestionar un frontend independiente. Para este proyecto, donde la interacción principal se basa en formularios, tablas y pantallas de gestión, Thymeleaf ofrece una solución suficiente, clara y mantenible.

Thymeleaf Layout Dialect

Además de Thymeleaf, se ha utilizado **Thymeleaf Layout Dialect** para organizar mejor las plantillas HTML. Esta tecnología permite definir una plantilla base común y reutilizarla en las distintas páginas de la aplicación.

En el proyecto, esto permite tener una estructura común con elementos como el layout principal, el menú lateral, la barra superior y los mensajes del sistema. De esta forma, las páginas concretas solo tienen que definir su contenido específico, mientras que la estructura general se mantiene centralizada.

Este enfoque mejora el mantenimiento del frontend. Si en el futuro se quiere modificar el diseño general de la aplicación, no es necesario cambiar todos los archivos HTML uno por uno, sino únicamente los fragmentos o plantillas comunes.

La elección de esta herramienta frente a duplicar código HTML en cada vista responde a una necesidad clara de organización. En una aplicación con varias pantallas para administración y cliente, reutilizar componentes visuales es fundamental para evitar errores, inconsistencias y trabajo repetido.

Thymeleaf Extras Spring Security

Para integrar la seguridad con las vistas se ha utilizado **Thymeleaf Extras Spring Security**. Esta extensión permite mostrar u ocultar contenido HTML en función del rol o estado de autenticación del usuario.

En el proyecto existen distintos perfiles de usuario, como administrador y cliente. Por tanto, no todos los usuarios deben ver las mismas opciones ni acceder a las mismas funcionalidades. Esta librería permite controlar desde las plantillas qué elementos deben mostrarse según el rol del usuario autenticado.

Por ejemplo, determinadas acciones administrativas solo deben estar disponibles para usuarios con rol de administrador, mientras que los clientes únicamente deben acceder a sus propias solicitudes, expedientes o documentos.

Esta tecnología se ha elegido porque se integra de forma directa con Spring Security y Thymeleaf, evitando tener que controlar manualmente en cada vista qué debe mostrarse o no.



Bootstrap

Para el diseño visual de la interfaz se ha utilizado **Bootstrap 5**. Bootstrap es un framework CSS que proporciona un sistema de rejilla, componentes visuales y utilidades para construir interfaces responsive de forma rápida y ordenada.

En el proyecto se ha utilizado para estructurar formularios, tarjetas, tablas, botones, columnas, márgenes, espaciados y diseño general de las páginas. Gracias a Bootstrap, la aplicación mantiene una apariencia coherente y se adapta mejor a distintos tamaños de pantalla.

Uno de los motivos principales para elegir Bootstrap es que acelera el desarrollo de interfaces. En lugar de diseñar desde cero todos los elementos visuales, Bootstrap proporciona clases preparadas para construir una interfaz limpia, funcional y consistente.

Bootstrap también resulta muy adecuado para aplicaciones de gestión, ya que este tipo de sistemas suelen necesitar paneles, formularios, listados, botones de acción y mensajes visuales. Todos estos elementos pueden construirse de forma clara con las utilidades que ofrece el framework.

Se ha utilizado fundamentalmente la distribución de elementos en pantalla, para estilos de botones, tarjetas (badges), iconos, etc.

Frente a desarrollar todo el CSS manualmente, Bootstrap permite ahorrar tiempo y reducir inconsistencias visuales. Frente a alternativas como Tailwind CSS, Bootstrap ofrece componentes más preparados desde el inicio y una curva de aprendizaje más sencilla para un proyecto de este tipo.



CSS personalizado

Además de Bootstrap, se ha utilizado **CSS personalizado** para adaptar el aspecto visual de la aplicación a las necesidades concretas del proyecto.

Bootstrap proporciona una base sólida, pero en muchos casos es necesario personalizar estilos para conseguir una identidad visual propia. Por este motivo, se han definido estilos específicos para elementos como tarjetas, cabeceras, menús, paneles, badges, estados, botones o distribución de contenido.

El CSS personalizado permite que la aplicación no tenga un aspecto genérico y que las pantallas mantengan una coherencia visual adaptada al sistema de gestión documental. También permite mejorar la experiencia de usuario, destacando información importante como estados de solicitudes, expedientes, incidencias o mensajes de confirmación.

La combinación de Bootstrap y CSS propio resulta equilibrada: Bootstrap aporta estructura y rapidez, mientras que el CSS personalizado permite ajustar detalles visuales y dar personalidad al proyecto.



HTML5

La estructura de las vistas se ha desarrollado mediante **HTML5**. HTML es la base de cualquier aplicación web, ya que permite definir el contenido y la estructura de las páginas.

En el proyecto, HTML se utiliza junto con Thymeleaf para construir formularios, listados, paneles de control, vistas de detalle, menús y componentes reutilizables. La utilización de HTML5 permite crear una estructura semántica, compatible con navegadores modernos y adecuada para una aplicación web actual.

HTML5 se ha elegido porque es el estándar actual para la construcción de interfaces web. Además, su integración con Thymeleaf permite generar contenido dinámico manteniendo una estructura clara y fácil de entender.



JavaScript

También se ha utilizado **JavaScript** para añadir comportamiento dinámico en determinadas partes de la interfaz. Aunque la aplicación se basa principalmente en renderizado desde servidor, algunas funcionalidades requieren interacción en el navegador.

JavaScript permite mejorar la experiencia del usuario sin necesidad de recargar la página completa en todos los casos. En el proyecto se utiliza para tareas como gestionar elementos dinámicos en formularios, actualizar partes de la interfaz o mostrar información de forma más interactiva.

La elección de JavaScript se debe a que es el lenguaje estándar del navegador. No requiere instalar tecnologías adicionales y permite añadir pequeñas funcionalidades dinámicas de forma directa.

No se ha optado por un framework frontend completo como React, Angular o Vue porque el proyecto no lo necesitaba. La mayor parte de la lógica se resuelve correctamente desde el servidor con Spring MVC y Thymeleaf. Añadir un framework de frontend habría aumentado la complejidad sin aportar una ventaja proporcional para el alcance de la aplicación.



Chart.js

Para la representación gráfica de información en los paneles de control se ha utilizado **Chart.js**. Esta librería permite crear gráficos en JavaScript de forma sencilla y visual.

En una aplicación de gestión documental, los paneles o dashboards ayudan a interpretar rápidamente el estado general del sistema. Por ejemplo, pueden utilizarse gráficos para mostrar estadísticas de solicitudes, expedientes o estados de tramitación.

Chart.js se ha elegido porque es una librería ligera, fácil de integrar y suficiente para representar gráficos básicos. No requiere una configuración compleja y puede incluirse directamente en las vistas de la aplicación.

Frente a librerías más avanzadas como D3.js, Chart.js resulta más sencilla y adecuada para el proyecto. D3.js ofrece un nivel de personalización muy alto, pero también requiere mayor complejidad de desarrollo. Para gráficos sencillos de panel de control, Chart.js ofrece un equilibrio más adecuado entre facilidad de uso y resultado visual.

Se ha utilizado en los gráficos del dashboard, se plantea para el futuro el uso de más gráficos de diferentes métricas.



MySQL

Como sistema gestor de base de datos se ha utilizado **MySQL**. Se ha elegido MySQL porque es una base de datos relacional ampliamente utilizada, estable, gratuita y suficiente para las necesidades del proyecto.

La aplicación necesita almacenar información estructurada: usuarios, clientes, solicitudes, expedientes, documentos, interesados, incidencias, tipos de trámite y tipos de incidencia. Para este tipo de información, una base de datos relacional resulta muy adecuada, ya que permite definir tablas, claves primarias, claves externas, restricciones e índices.

MySQL permite mantener la integridad de los datos mediante relaciones entre tablas. Esto es especialmente importante en el proyecto, porque muchos datos dependen unos de otros. Por ejemplo, una solicitud pertenece a un cliente, un expediente puede generarse a partir de una solicitud, un documento puede asociarse a una solicitud o expediente, y una incidencia pertenece a un expediente.

Otro motivo para elegir MySQL es su facilidad de instalación y uso durante el desarrollo. Además, herramientas como MySQL Workbench permiten visualizar gráficamente el modelo de datos, crear diagramas y revisar las relaciones entre tablas, lo que resulta muy útil para la documentación del proyecto.

Frente a bases de datos NoSQL como MongoDB, MySQL se ha considerado más apropiado porque el sistema trabaja con datos muy relacionados y estructurados. Una base de datos documental podría ser útil para otros contextos, pero en este caso la necesidad de relaciones claras entre entidades hace que una base de datos relacional sea una mejor opción.

Frente a PostgreSQL, MySQL también ofrece una solución válida. PostgreSQL es más potente en algunos aspectos avanzados, pero MySQL resulta más sencillo, conocido y suficiente para el alcance del proyecto. Para una aplicación académica y empresarial de tamaño medio, MySQL ofrece un equilibrio adecuado entre facilidad de uso, rendimiento y estabilidad.



MySQL Workbench

Para el diseño y revisión del modelo de base de datos se ha utilizado **MySQL Workbench**. Esta herramienta permite visualizar tablas, relaciones, claves primarias, claves externas e índices de forma gráfica.

En el proyecto ha sido útil para representar el modelo relacional definitivo y comprobar cómo se conectan las distintas entidades de la aplicación.

MySQL Workbench se ha elegido porque se integra directamente con MySQL y permite trabajar tanto con la estructura de la base de datos como con consultas SQL. Frente a otras herramientas genéricas de diagramado, Workbench tiene la ventaja de representar el modelo real de la base de datos y no únicamente un dibujo conceptual.



remoto.



Git y GitHub

Para el control de versiones se ha utilizado Git, junto con GitHub como repositorio. Esta herramienta permite llevar un historial de los cambios realizados en el proyecto, recuperar versiones anteriores y trabajar desde distintos equipos de forma ordenada.

Git resulta especialmente útil en un proyecto que evoluciona progresivamente, ya que permite guardar avances mediante commits. Esto facilita identificar cuándo se ha añadido una funcionalidad, cuándo se ha corregido un error o cuándo se ha modificado una parte concreta del código.

GitHub, por su parte, permite almacenar el proyecto en la nube y sincronizarlo entre varios ordenadores. Esto resulta útil para continuar el desarrollo desde diferentes lugares sin depender de copias manuales o sistemas de sincronización menos adecuados para código fuente.

Frente a trabajar únicamente con carpetas locales o servicios como OneDrive, Git ofrece un control mucho más preciso sobre los cambios del código. Además, evita muchos problemas habituales al sincronizar proyectos de programación mediante herramientas pensadas para documentos generales.



IDE de desarrollo - IntelliJ

Para programar la aplicación se ha utilizado un entorno de desarrollo integrado, como **IntelliJ IDEA**. Este tipo de herramienta facilita la escritura, navegación, ejecución y depuración del código.

En un proyecto Java con Spring Boot, un IDE resulta especialmente útil porque permite detectar errores, autocompletar código, navegar entre clases, ejecutar la aplicación, revisar dependencias Maven y trabajar con Git desde la misma interfaz.

El uso de un IDE mejora la productividad y reduce errores durante el desarrollo. Frente a utilizar únicamente un editor de texto, un entorno como IntelliJ IDEA ofrece herramientas específicas para proyectos Java y Spring Boot, lo que facilita mucho el trabajo.

Resumen de la elección tecnológica

En conjunto, las tecnologías elegidas permiten construir una aplicación web robusta, mantenible y adecuada para un entorno de gestión documental.

Java y Spring Boot proporcionan una base sólida para el backend. Spring MVC permite estructurar la aplicación siguiendo el patrón Modelo-Vista-Controlador. Spring Data JPA e Hibernate facilitan la persistencia de datos y la relación entre entidades. MySQL ofrece una base de datos relacional estable y adecuada para la información estructurada del sistema. Thymeleaf y Bootstrap permiten construir una interfaz web clara y funcional. Spring Security aporta control de acceso y protección de rutas. PDFBox y Tess4J amplían el sistema con capacidades de

tratamiento documental y OCR. Maven, Git y GitHub facilitan la gestión técnica del proyecto.

La elección tecnológica se ha realizado buscando un equilibrio entre funcionalidad, estabilidad, facilidad de aprendizaje y capacidad de ampliación futura. Aunque existían otras alternativas posibles, el conjunto formado por Java, Spring Boot, MySQL y Thymeleaf resulta especialmente adecuado para una aplicación de gestión documental interna, donde se priorizan la seguridad, la organización del código, la persistencia de datos y la facilidad de mantenimiento.

También ha tenido peso en la elección el conocimiento de dichos lenguajes y herramientas frente a otros de los cuales tengo menos conocimientos y experiencia, esta elección ha favorecido mucho al desarrollo ya que he podido avanzar mucho más rápido y he podido dedicar más tiempo al desarrollo de la lógica de la aplicación.

IMPLEMENTACIÓN DEL SISTEMA

Estructura del proyecto

La aplicación se ha desarrollado utilizando el framework Spring Boot y siguiendo una arquitectura en capas. Esta organización permite separar las diferentes responsabilidades del sistema y facilita el mantenimiento, la ampliación y la comprensión del código. En lugar de concentrar toda la lógica en una única clase o componente, el proyecto se divide en paquetes especializados, de forma que cada uno de ellos cumple una función concreta dentro de la aplicación.

La estructura principal del proyecto se organiza en los siguientes paquetes:

- **model:** contiene las entidades principales del sistema, mapeadas con la base de datos mediante JPA.
- **repository:** contiene las interfaces encargadas del acceso a datos mediante Spring Data JPA.
- **service:** define las operaciones de negocio disponibles en la aplicación.
- **service.impl:** contiene la implementación concreta de los servicios.
- **controller:** contiene los controladores MVC encargados de gestionar las peticiones web.
- **security:** contiene las clases relacionadas con la autenticación, autorización y redirección de usuarios.

- **config:** contiene clases de configuración auxiliar utilizadas por la aplicación.
- **dto:** contiene objetos auxiliares empleados para transportar información entre formularios, vistas y servicios.
- **enums:** contiene enumeraciones utilizadas para representar valores controlados del sistema.
- **resources/templates:** contiene las vistas HTML desarrolladas con Thymeleaf.
- **resources/static:** contiene recursos estáticos como hojas de estilo, imágenes y archivos JavaScript.

Esta estructura permite aplicar el principio de separación de responsabilidades. Los controladores se encargan de recibir las peticiones realizadas desde el navegador, los servicios contienen la lógica de negocio, los repositorios gestionan el acceso a la base de datos y las entidades representan el modelo de datos de la aplicación.

Gracias a esta división, el proyecto resulta más ordenado y mantenible. Si en el futuro fuera necesario modificar una funcionalidad concreta, añadir una nueva entidad o cambiar una regla de negocio, el cambio podría realizarse en la capa correspondiente sin afectar de forma innecesaria al resto de la aplicación.

Entidades del sistema (Entities)

Las entidades representan los objetos principales del sistema y se corresponden con las tablas de la base de datos. Cada entidad se implementa como una clase Java anotada con `@Entity`, lo que permite a JPA mapear automáticamente dicha clase con su tabla correspondiente.

Entre las principales entidades del sistema se encuentran Cliente, Usuario, Solicitud, Expediente, Documento, Incidencia, Interesado, ExpedienteInteresado, TipoTramite, TipoIncidencia, Historial y Mensaje.

La entidad **Cliente** representa a los clientes o empresas que utilizan la plataforma. Cada cliente puede tener usuarios asociados y puede estar vinculado a distintas solicitudes y expedientes. De esta forma, el sistema permite separar correctamente la información perteneciente a cada cliente.

La entidad **Usuario** representa a las personas que acceden a la aplicación. Cada usuario dispone de credenciales de acceso y de un rol determinado dentro del sistema. Esta entidad resulta fundamental para controlar la autenticación, la

autorización y la relación entre los usuarios y los datos que pueden consultar o gestionar.

La entidad **Solicitud** representa las peticiones iniciales realizadas por los clientes. A través de una solicitud, el cliente puede iniciar un trámite, indicar los datos principales y aportar documentación. Esta entidad permite registrar la fase inicial del proceso antes de que el administrador revise la información y, si procede, genere un expediente. En cuanto a los interesados, durante el desarrollo se tomó la decisión de guardarlos como campos de esta tabla y solo hacer las validaciones al hacer la conversión a expedientes, para evitar guardar personas en la base de datos hasta confirmar que una solicitud pasa a expediente.

La entidad **Expediente** representa el núcleo principal del trámite administrativo. Contiene información como el cliente asociado, el tipo de trámite, el estado del expediente, la matrícula del vehículo, las observaciones y la relación con otros elementos del sistema. Un expediente puede estar vinculado a documentos, incidencias e interesados, lo que permite agrupar toda la información necesaria para realizar el seguimiento completo del trámite.

La entidad **Documento** permite registrar los archivos subidos a la plataforma. Aunque el archivo físico se almacena en el sistema de ficheros del servidor, en la base de datos se guardan sus metadatos principales, como el nombre original, el nombre interno, el tipo de documento, la fecha de subida, el usuario que lo subió y la solicitud o expediente al que pertenece.

La entidad **Incidencia** permite registrar problemas, avisos o situaciones pendientes dentro de un expediente. Gracias a esta entidad, el administrador puede dejar constancia de documentación incorrecta, información incompleta o cualquier aspecto que deba ser revisado. Esta funcionalidad mejora el seguimiento interno de los trámites.

La entidad **Interesado** representa a las personas relacionadas con un expediente, como titulares, compradores, vendedores, representantes u otros participantes. Para representar correctamente la relación entre expedientes e interesados se utiliza la entidad ExpedienteInteresado, que actúa como tabla intermedia. Esta solución permite que un expediente tenga varios interesados y que cada interesado desempeñe un rol concreto dentro del trámite.

Las entidades **TipoTramite** y **TipoIncidencia** permiten normalizar información que podría repetirse en varias partes del sistema. En lugar de almacenar estos valores como texto libre, se gestionan como entidades independientes, lo que facilita su mantenimiento y permite ampliar el sistema en el futuro de forma más ordenada.

Además, el sistema durante el desarrollo se han incorporado las entidades **Historial** y **Mensaje**. La entidad Historial permite registrar acciones relevantes

realizadas dentro de la aplicación, mejorando la trazabilidad del sistema. Gracias a ella, es posible conservar información sobre cambios, operaciones o eventos importantes asociados a los trámites. Por su parte, la entidad Mensaje permite dar soporte a funcionalidades de comunicación interna entre usuarios, centralizando dentro de la plataforma la información relacionada con solicitudes, expedientes o incidencias.

El uso de entidades permite trabajar con objetos Java dentro de la aplicación, mientras que JPA e Hibernate se encargan de traducir estas operaciones al modelo relacional de la base de datos.

Repositorios

La capa de acceso a datos se implementa mediante repositorios. Estos repositorios son interfaces que extienden `JpaRepository`, proporcionado por Spring Data JPA. Gracias a esta tecnología, la aplicación puede realizar operaciones sobre la base de datos sin necesidad de escribir manualmente la mayoría de las consultas SQL.

Cada entidad principal dispone de su propio repositorio, encargado de gestionar las operaciones de persistencia relacionadas con dicha entidad. A través de estos repositorios se pueden realizar operaciones habituales como guardar registros, buscar por identificador, obtener listados, eliminar registros o comprobar si determinados datos ya existen.

Además de las operaciones básicas, los repositorios también permiten definir métodos de búsqueda personalizados. En el proyecto se utilizan para recuperar información específica según las necesidades de la aplicación, como obtener expedientes pertenecientes a un cliente, localizar solicitudes asociadas a un usuario, consultar documentos vinculados a una solicitud o expediente, buscar interesados por sus datos identificativos o recuperar incidencias asociadas a un expediente concreto.

Esta capa permite centralizar el acceso a la base de datos y evita que los controladores o servicios tengan que trabajar directamente con consultas SQL. De esta forma, el código resulta más limpio, más mantenible y fácil de adaptar en caso de que sea necesario modificar alguna consulta o relación del modelo de datos.

Servicios y lógica de negocio

La capa de servicios contiene la lógica de negocio de la aplicación. Los servicios actúan como intermediarios entre los controladores y los repositorios, coordinando las operaciones necesarias para cumplir las funcionalidades del sistema.

En esta capa se implementan las reglas que determinan cómo deben procesarse los datos dentro de la aplicación. Los controladores no acceden directamente a la base de datos, sino que delegan el trabajo en los servicios correspondientes. Esto permite mantener una estructura más limpia y evita que la lógica de negocio quede repartida entre diferentes controladores.

Uno de los procesos más representativos de esta capa es la creación de un expediente a partir de una solicitud. Esta operación no consiste únicamente en insertar un nuevo registro en la base de datos, sino que implica recuperar la solicitud original, crear el expediente correspondiente, mantener la relación entre ambos elementos, asociar el cliente y el tipo de trámite, conservar la trazabilidad del proceso y actualizar el estado de la solicitud para indicar que ya ha sido tramitada.

En cuanto a los tipos de estado que puede tener un trámite también se han creado diversas validaciones para prevenir por ejemplo que un trámite con incidencias activas cambie de estado, o cambiar de estado un trámite que ya ha sido declarado como finalizado, entre otras lógicas de negocio.

También destaca la gestión de interesados. El sistema permite asociar varios interesados a un mismo expediente e indicar el rol que desempeña cada uno dentro del trámite. También el sistema es capaz de comprobar si un interesado está registrado y recuperar los datos. Esta lógica se centraliza en los servicios para evitar duplicidades y garantizar que los datos se almacenen de forma coherente.

Otro servicio importante es el encargado de la gestión documental. Este servicio no solo guarda archivos, sino que también determina si el documento pertenece a una solicitud o a un expediente, genera nombres internos únicos para evitar sobreescrituras, almacena los archivos en el sistema de ficheros y registra sus metadatos en la base de datos. Además, interviene en las operaciones de consulta, descarga, visualización y eliminación de documentos.

La aplicación también incluye servicios específicos para el procesamiento de documentos PDF y la aplicación de OCR. Estos servicios permiten analizar expedientes completos, extraer texto de sus páginas, detectar tipos documentales y dividir automáticamente un único PDF en varios documentos independientes.

Centralizar estas operaciones en la capa de servicios permite que la lógica de negocio sea reutilizable, más fácil de probar y más sencilla de mantener.

Controladores MVC

Los controladores constituyen el punto de entrada de la aplicación para las peticiones realizadas desde el navegador. En este proyecto se utilizan

controladores MVC anotados con `@Controller`, ya que la aplicación genera vistas HTML mediante Thymeleaf desde el lado del servidor.

Los controladores se encargan de recibir las peticiones HTTP, procesar los datos enviados desde los formularios, invocar a los servicios correspondientes y devolver la vista que debe mostrarse al usuario. De esta forma, actúan como enlace entre la interfaz de usuario y la lógica interna de la aplicación.

En el proyecto existen controladores diferenciados según la zona funcional de la aplicación. Los controladores relacionados con el área de administración permiten consultar solicitudes, crear expedientes, revisar documentación, gestionar incidencias y acceder a listados generales del sistema. Por otro lado, los controladores del área de cliente permiten crear nuevas solicitudes, consultar expedientes propios y aportar documentación.

También existe un controlador específico para documentos, encargado de gestionar acciones como la subida de archivos, la visualización, la descarga y la eliminación de documentos. Esta separación permite que cada controlador tenga una responsabilidad concreta y evita concentrar demasiada lógica en una única clase.

Los controladores no contienen lógica de negocio compleja. Su función principal es coordinar la petición recibida, preparar los datos necesarios para la vista y delegar las operaciones importantes en la capa de servicios.

Gestión de la seguridad

La seguridad de la aplicación se ha implementado mediante Spring Security. Esta tecnología permite controlar la autenticación de usuarios, la autorización según roles y el acceso a las diferentes zonas del sistema.

El sistema utiliza autenticación basada en usuario y contraseña. Antes de acceder a la aplicación, cada usuario debe iniciar sesión con sus credenciales. Una vez autenticado, el sistema determina qué funcionalidades puede utilizar en función del rol asignado.

En el proyecto se han definido principalmente dos roles: CLIENTE y ADMIN. El rol CLIENTE permite acceder al área privada del cliente, consultar sus solicitudes y expedientes, y subir documentación. El rol ADMIN permite acceder a la zona de administración, gestionar solicitudes, crear expedientes, registrar incidencias, revisar documentos y realizar el seguimiento global del sistema.

La configuración de seguridad se encuentra centralizada en una clase específica, donde se definen las rutas públicas, las rutas protegidas y los permisos necesarios para acceder a cada zona. Las rutas de administración quedan

restringidas a usuarios con rol de administrador, mientras que las rutas de cliente quedan limitadas a usuarios con rol de cliente.

Además, las contraseñas no se almacenan en texto plano en la base de datos, sino cifradas mediante un codificador seguro. Esta decisión mejora la protección del sistema, ya que impide que las contraseñas reales puedan consultarse directamente, aunque alguien tuviera acceso a la base de datos.

La aplicación también utiliza un servicio personalizado de carga de usuarios, encargado de recuperar la información del usuario desde la base de datos durante el proceso de autenticación. Tras iniciar sesión correctamente, el sistema redirige al usuario a una zona u otra en función de su rol, permitiendo que cada perfil acceda directamente a su panel correspondiente.

Gestión de solicitudes y expedientes

Una de las funcionalidades principales de la aplicación es la gestión de solicitudes y expedientes. El sistema diferencia entre ambos conceptos para representar correctamente el flujo de trabajo de la gestoría.

La solicitud representa la petición inicial realizada por el cliente. A través de ella, el usuario puede comunicar que necesita iniciar un trámite, indicar los datos básicos y adjuntar la documentación correspondiente. Esta solicitud queda registrada en el sistema y puede ser revisada posteriormente por un administrador.

El expediente representa una fase más avanzada del proceso. Cuando el administrador revisa una solicitud y decide iniciar formalmente el trámite, puede crear un expediente asociado. De esta forma, se mantiene la trazabilidad entre la petición original del cliente y el expediente administrativo generado a partir de ella.

Esta separación permite organizar mejor el trabajo interno. Las solicitudes funcionan como punto de entrada de nuevos trámites, mientras que los expedientes agrupan toda la información necesaria para realizar el seguimiento completo: documentos, interesados, incidencias, mensajes, historial y estado del trámite.

El sistema también permite filtrar y consultar solicitudes o expedientes según diferentes criterios, como estado, tipo de trámite o matrícula. Esto facilita el trabajo diario del administrador y permite localizar rápidamente la información necesaria.

Gestión de interesados

La gestión de interesados se ha incorporado para representar a las personas que intervienen en un expediente. En trámites relacionados con vehículos es habitual que participen distintas personas con roles diferentes, como comprador, vendedor, titular, representante o autorizado.

Para modelar esta situación, el sistema utiliza una entidad específica de relación entre expediente e interesado. Esta entidad intermedia permite asociar varios interesados a un mismo expediente e indicar el rol que desempeña cada uno de ellos.

Esta solución resulta más flexible que guardar los datos de los interesados directamente dentro del expediente, ya que evita duplicidades y permite representar correctamente relaciones más complejas. Además, facilita futuras ampliaciones, como reutilizar datos de interesados, añadir nuevos roles o mejorar la consulta de personas vinculadas a distintos trámites.

Gestión de documentos

La gestión documental es uno de los elementos fundamentales del sistema. La aplicación permite subir, almacenar, consultar, visualizar, descargar y eliminar documentos asociados a solicitudes o expedientes.

Cuando un usuario sube un documento, el archivo físico se almacena en el sistema de ficheros del servidor. En la base de datos no se guarda el contenido completo del archivo, sino únicamente sus metadatos. Entre estos metadatos se incluyen el nombre original del archivo, el nombre interno utilizado para almacenarlo, el tipo de documento, la fecha de subida, el usuario que ha realizado la carga y la solicitud o expediente al que pertenece.

Este enfoque permite mantener la base de datos más ligera y facilita la gestión de archivos de mayor tamaño. Además, separar el almacenamiento físico del archivo y el registro de sus metadatos permite organizar mejor la información y acceder a los documentos de forma controlada.

La aplicación genera nombres internos únicos para los archivos subidos. De esta manera se evita que dos documentos con el mismo nombre puedan sobrescribirse en el servidor. Al mismo tiempo, el nombre original se conserva para mostrarlo al usuario, manteniendo una experiencia más clara y comprensible.

El sistema permite asociar documentos tanto a solicitudes como a expedientes. Esta decisión responde al flujo real de trabajo de la aplicación: inicialmente el cliente puede aportar documentación al crear una solicitud, y posteriormente el administrador puede seguir gestionando documentos dentro del expediente ya creado.

Además, la aplicación incorpora controles de acceso sobre los documentos. Antes de permitir la visualización o descarga de un archivo, el sistema comprueba que el usuario autenticado tiene permiso para acceder a la solicitud o expediente al que pertenece dicho documento. Esto evita que un cliente pueda acceder a documentación de otro cliente y refuerza la seguridad del sistema.

Procesamiento OCR y división automática de PDFs

Una de las funcionalidades más avanzadas de la aplicación es el procesamiento automático de documentos PDF mediante OCR. Esta funcionalidad se ha incorporado para facilitar la gestión de expedientes completos, que habitualmente pueden contener varios documentos administrativos agrupados en un único archivo.

El proceso comienza cuando el usuario sube un PDF indicando que se trata de un expediente completo. A partir de ese momento, el sistema procesa el archivo página por página. Cada página se analiza para intentar extraer el texto que contiene. Para ello se utiliza un sistema OCR basado en Tesseract, integrado en Java mediante Tess4J, junto con herramientas de tratamiento de PDF como Apache PDFBox.

El texto obtenido se analiza mediante una serie de reglas y patrones de identificación. El objetivo es detectar automáticamente el tipo de documento presente en cada página, como DNI, contrato de compraventa, factura, permiso de circulación, ficha técnica, mandato de representación u otros documentos habituales en trámites administrativos de vehículos.

Cuando el sistema detecta que varias páginas consecutivas pertenecen al mismo tipo documental, las agrupa dentro de un mismo bloque. Si detecta un tipo diferente, inicia un nuevo bloque documental. Posteriormente, el sistema divide el PDF original en varios archivos independientes, cada uno correspondiente a un documento detectado.

Cada documento generado se almacena en el sistema de ficheros y se registra en la base de datos con sus metadatos correspondientes. De esta forma, un único expediente completo puede transformarse automáticamente en varios documentos clasificados y asociados correctamente a la solicitud o expediente.

Esta funcionalidad aporta un valor añadido importante al proyecto, ya que reduce el trabajo manual del administrador. En lugar de revisar un PDF completo, separar manualmente las páginas y clasificar cada documento, el sistema automatiza parte de este proceso y mejora la organización documental.

También se implementó un sistema de edición de documentos dentro de la propia plataforma, que permite ajustar el tipo de documento, cambiar el nombre, incluso extraer páginas del documento para separarlas y corregir

posibles clasificaciones erróneas. Los documentos no siempre son de buena calidad y las lecturas son complicadas muchas veces debido a la gran variedad de documentos que se manejan. Con este sistema se completa y mejora notablemente la gestión documental, permitiendo corregir posibles errores derivados del procesamiento automático.

Integración con la base de datos

La comunicación entre la aplicación y la base de datos se realiza mediante Spring Data JPA e Hibernate. Estas tecnologías permiten mapear las entidades Java con las tablas de la base de datos y trabajar con objetos dentro del código, mientras el sistema se encarga de traducir esas operaciones a consultas SQL.

El modelo de datos utiliza relaciones entre entidades para reflejar la lógica real de la aplicación. Por ejemplo, un cliente puede tener varios usuarios, una solicitud pertenece a un cliente, un expediente puede generarse a partir de una solicitud, un expediente puede contener varios documentos e incidencias, y un expediente puede relacionarse con varios interesados mediante una entidad intermedia.

Además, el sistema utiliza enumeraciones para representar valores controlados, como los estados de solicitudes, estados de expedientes, roles de usuario, roles de interesados y tipos de documento. Esto permite limitar los valores posibles, reducir errores y mejorar la coherencia de los datos almacenados.

El uso de JPA simplifica el desarrollo porque evita tener que escribir manualmente la mayoría de las consultas SQL necesarias para las operaciones habituales. No obstante, el diseño de la base de datos sigue siendo relacional, con claves primarias, claves externas y relaciones entre tablas que garantizan la integridad de la información.

Interfaz de usuario

La interfaz de usuario se ha desarrollado mediante HTML, Thymeleaf, Bootstrap y CSS personalizado. Thymeleaf permite generar páginas dinámicas desde el servidor, integrando los datos obtenidos desde los controladores con las vistas HTML.

La aplicación cuenta con diferentes pantallas según el tipo de usuario. Los clientes disponen de un panel propio desde el que pueden consultar sus expedientes, crear solicitudes y subir documentación. Los administradores, por su parte, acceden a una zona de gestión desde la que pueden revisar solicitudes, crear expedientes, consultar listados, gestionar documentos, registrar incidencias y realizar el seguimiento de los trámites.

Bootstrap se ha utilizado para estructurar la interfaz mediante tarjetas, tablas, formularios, botones, columnas y elementos responsive. Además, se ha añadido CSS personalizado para adaptar el diseño visual a las necesidades concretas del proyecto y conseguir una apariencia más cuidada y coherente.

La interfaz se ha diseñado buscando claridad y facilidad de uso. Al tratarse de una aplicación de gestión, se ha dado prioridad a que la información esté bien organizada, que las acciones principales sean accesibles y que los usuarios puedan identificar fácilmente el estado de solicitudes, expedientes, documentos e incidencias.

Aunque durante la fase de diseño se elaboraron mockups iniciales para definir la estructura general de la aplicación, durante la implementación fue necesario adaptar varias pantallas al funcionamiento real del sistema. Estas modificaciones surgieron al incorporar nuevas funcionalidades, como la gestión de incidencias, el historial de cambios, la mensajería interna y la gestión avanzada de documentos PDF.

Por este motivo, el diseño final no coincide exactamente con los mockups originales. Sin embargo, esta evolución se considera positiva, ya que permitió construir una interfaz más completa, más clara y mejor adaptada al uso real de la aplicación.

Pantalla de inicio de sesión

En esta pantalla prácticamente se mantiene el diseño original, adaptado simplemente colores y algún ajuste, se ha optado por un fondo un poco más oscuro y campos un poco más visibles.

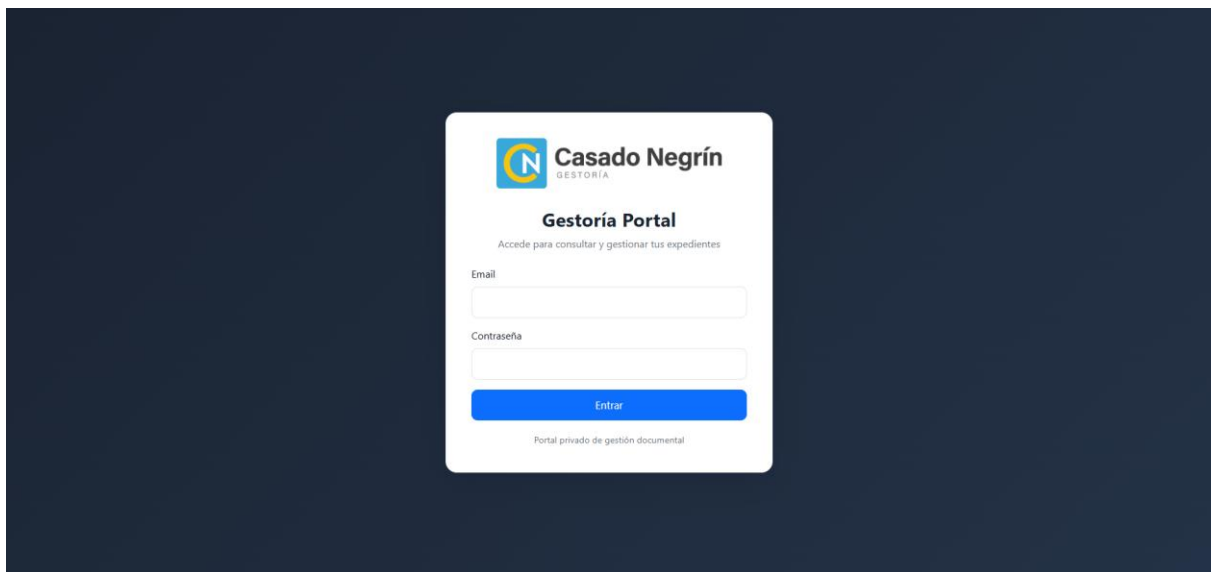


Ilustración 17. Diseño final - Login

Panel principal del cliente (Dashboard cliente)

Para el dashboard del cliente se ha optado por incluir colores en función de los estados para los recuadros informativos del número de trámites pendientes y se ha optado también por la incorporación de gráficos para un aspecto más visual y moderno. Se ha incluido también una tabla que incluye los últimos 5 expedientes / solicitudes para darle algo más de contenido a la página.

También se mantiene la parte con accesos directos a otros puntos del sistema tanto en el panel principal como en un menú lateral.

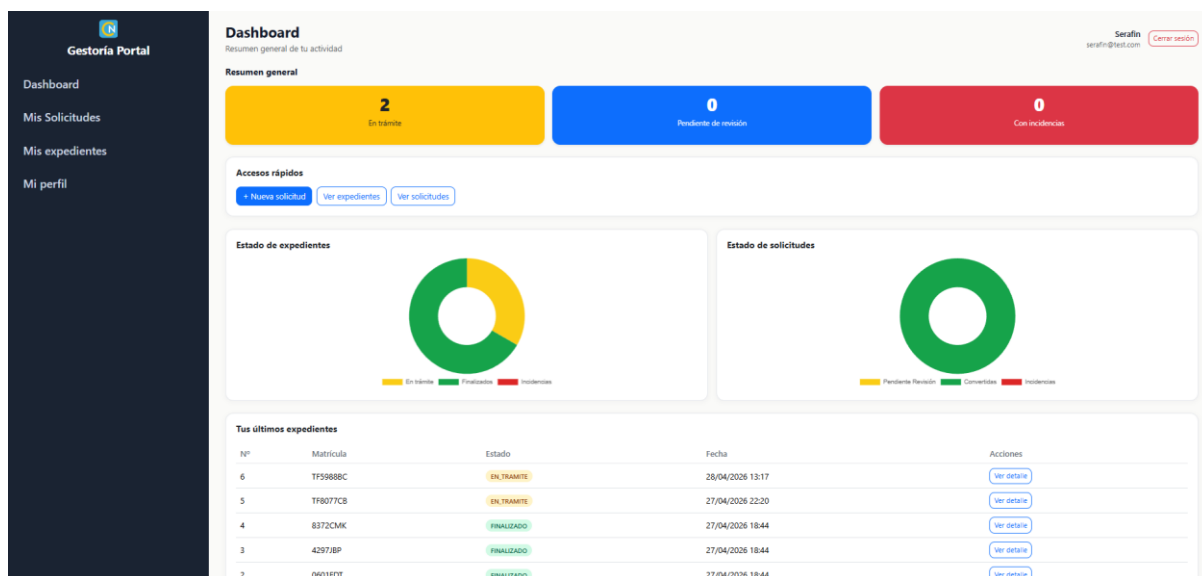


Ilustración 18. Diseño final – Dashboard cliente

Listado de expedientes

Al igual que el resto de la interfaz, se han ajustado los colores y añadido algunas opciones más de filtrado, manteniendo la misma estructura base inicialmente planteada. Se ha decidido eliminar el número de documentos de los trámites al no aportar información útil y se ha incorporado la fecha de última modificación y el usuario que la realizó, para tener información directa de la trazabilidad del expediente directamente en el listado.

Ahora se puede filtrar por estado y tipo de trámite, también se ha añadido un campo de búsqueda por matrícula del vehículo. También se ha diseñado para que la lista se ordene por antigüedad, mostrándose en la parte superior los trámites más recientes en función de su última modificación. Con el botón de detalle se accede directamente a la información de cada expediente.

Mis expedientes

Mis expedientes

Serafín
serafin@test.com [Cerrar sesión](#)

Estado: Todos Tipo de trámite: Todos Matriculación: Buscar matriculación [Filtrar](#)

ID	Tipo trámite	Matriculación	Estado	Fecha creación	Última modificación	Acciones
6	TRASPASO	TF5988BC	EN TRAMITE	28/04/2026 13:17	28/04/2026 23:02	Ver detalle
5	TRASPASO	TF8077CB	EN TRAMITE	27/04/2026 22:20	27/04/2026 22:20	Ver detalle
4	TRASPASO	8372CMK	FINALIZADO	27/04/2026 18:44	27/04/2026 18:45	Ver detalle
3	TRASPASO	4297BP	FINALIZADO	27/04/2026 18:44	27/04/2026 18:45	Ver detalle
2	TRASPASO	0601FDI	FINALIZADO	27/04/2026 18:44	27/04/2026 18:45	Ver detalle
1	TRASPASO	1146IBG	FINALIZADO	27/04/2026 17:10	27/04/2026 18:44	Ver detalle

Ilustración 19. Diseño final - Lista expedientes (cliente)

Detalle de expediente

Se ha puesto especial dedicación a la elaboración de esta interfaz, al ser uno de los puntos claves de la aplicación. En la fase de implementación se añadieron dos nuevas funcionalidades, el historial del expediente y el servicio de mensajería dentro del expediente para comunicación de los usuarios.

Se le ha dado un aspecto muy vistoso aprovechando mucho los colores para resaltar los elementos importantes y la introducción de pequeños iconos según el tipo de información de cada campo.

El cuadro de historial muestra los cambios relevantes dentro del expediente como cambios de estado, subida de documentación e incidencias.

Se incorpora un cuadro cuando figura alguna incidencia en el trámite, remarcado en un color rojo muy destacado para mayor impacto visual, dentro figura el tipo de incidencia y la fecha, así como el administrador que la ha creado/resuelto. Una vez resuelta el recuadro se vuelve de color verde para mostrar que el expediente puede continuar con la tramitación.

En cuanto a los documentos, se mantiene el formato de tipo listado con todos los documentos aportados, el tipo de documento, usuario que lo aporta y la fecha. También un pequeño menú con las opciones disponibles.

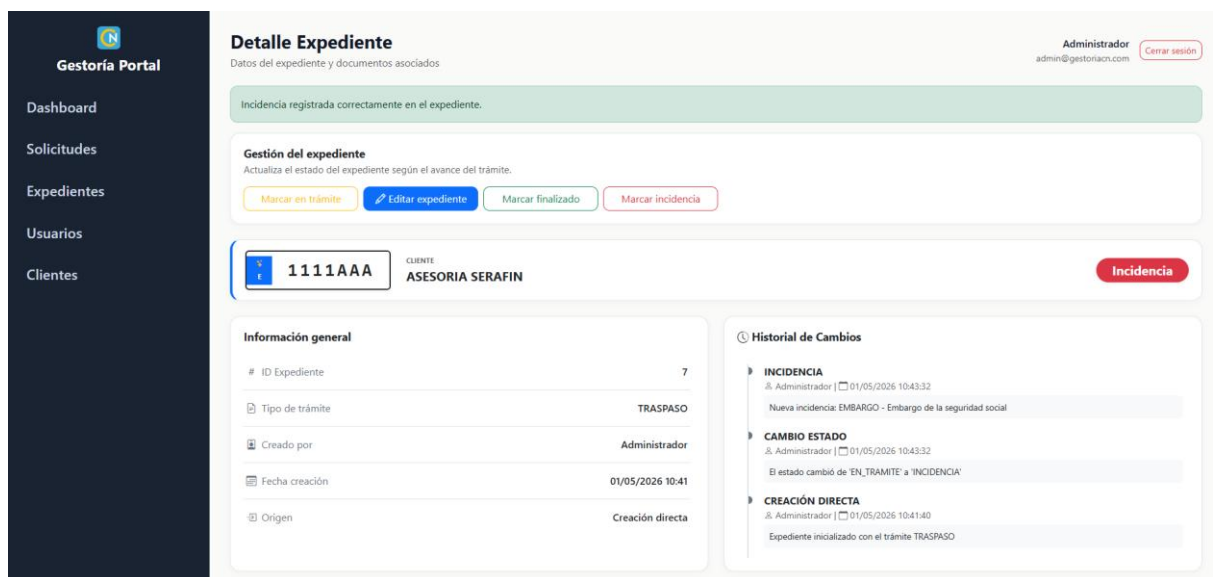


Ilustración 20. Diseño final – Detalle expediente (1/2)

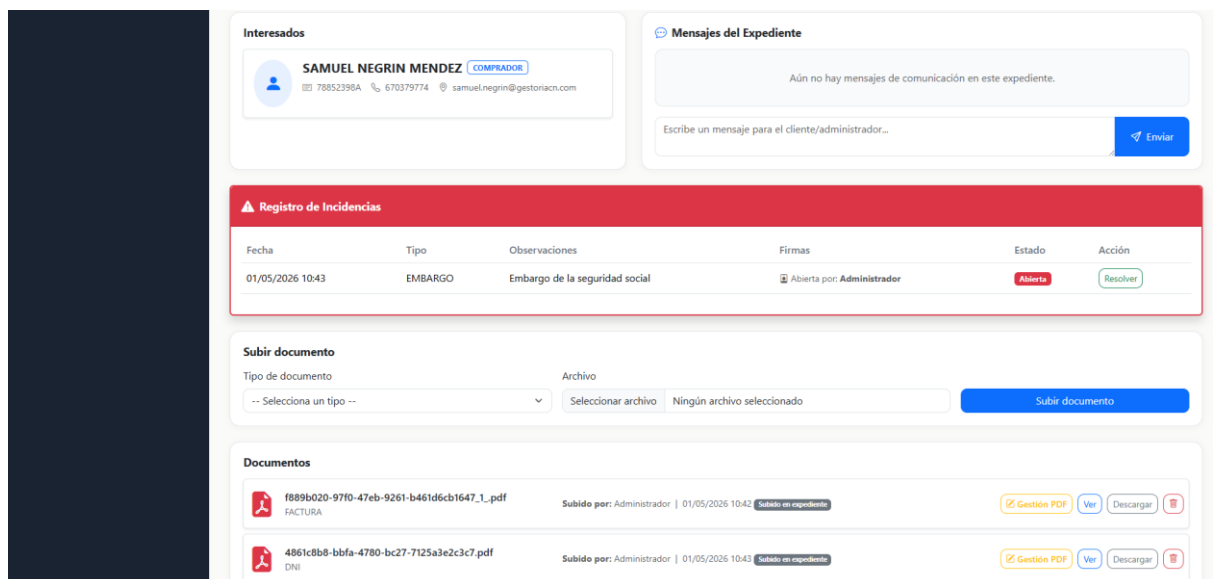


Ilustración 21. Diseño final – Detalle expediente (2/2)

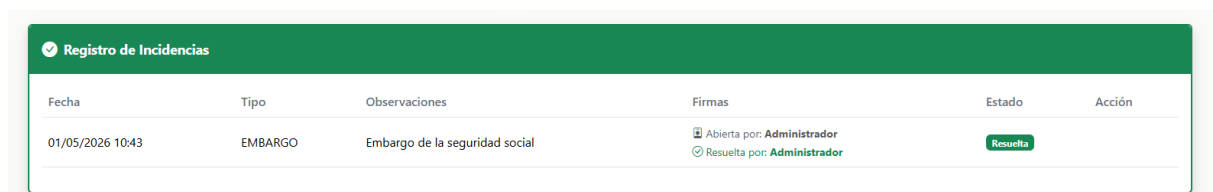


Ilustración 22. Diseño final – Incidencia resuelta

Otro elemento importante que se ha añadido es una pantalla de carga cuando se suben documentos a procesar por el motor OCR que puede tardar unos 15-30 segundos en función del tamaño del documento.

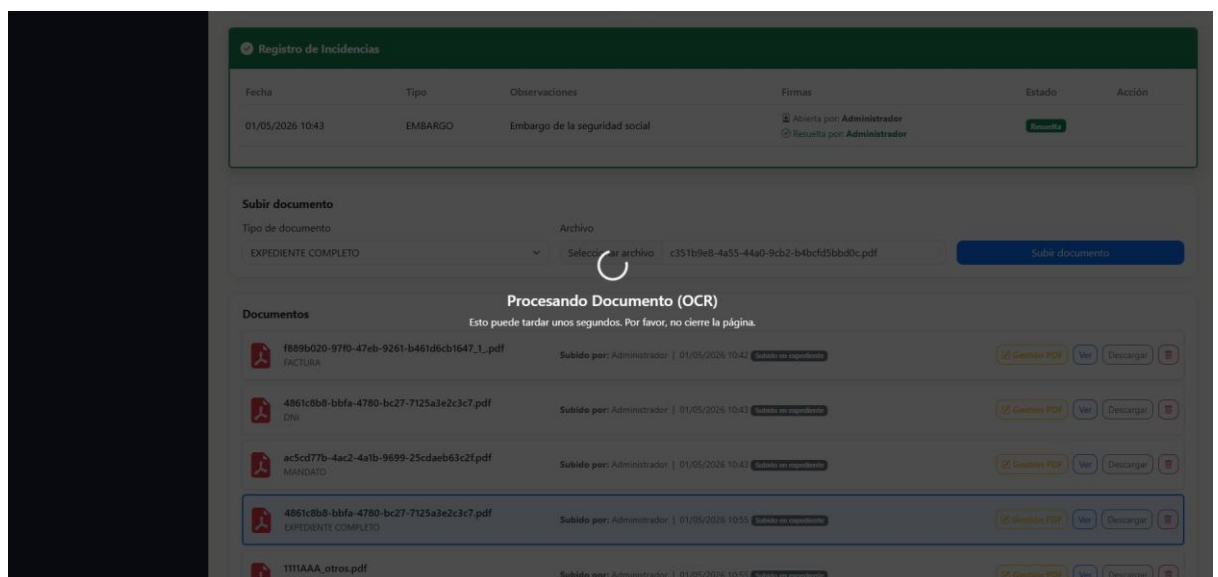


Ilustración 23. Diseño final – Información de procesamiento

Creación de solicitud / expediente

Para la creación de nuevos trámites se ha optado por una fórmula muy simple, con pocos datos obligatorios para que el cliente los rellene de la forma más rápida posible. En un futuro se estudiará la incorporación de nuevos campos o de más información tanto del vehículo como de la operación que se lleva a cabo.

De momento para esta primera etapa inicial se ha optado por esta versión sencilla para fomentar el uso por parte de los clientes y evitar ese posible rechazo inicial a rellenar demasiados datos de forma obligatoria.

Tratamos así que el uso de aplicación genere una mayor satisfacción y que no piensen que están “haciendo el trabajo de la gestoría” cumplimentando más datos de los necesarios y de esta manera prefieran mantener el uso de canales tradicionales para no complicarse en exceso.

The screenshot shows the 'Nuevo Expediente' (New Document) form. It has a dark sidebar with the 'Gestoría Portal' logo and navigation links: Dashboard, Solicitudes, Expedientes, Usuarios, and Clientes. The main form area is titled 'Nuevo Expediente' and 'Registro de nuevo expediente'. It includes a 'Crear expediente' section with fields for 'Cliente *' (dropdown), 'Tipo de trámite *' (dropdown), and 'Matrícula *' (text). Below this is an 'Interesado 1 (opcional)' section with fields for 'Rol' (dropdown), 'Nombre' (text), 'DNI/NIF' (text), 'Teléfono' (text), and 'Dirección' (text). At the bottom is an 'Interesado 2 (opcional)' section. The form is light-themed with a white background.

Ilustración 24. Diseño final – Nuevo trámite (1/2)

Ilustración 25. Diseño final – Nuevo trámite (2/2)

Despliegue de la aplicación

Una vez finalizada la fase principal de desarrollo de la aplicación, se procedió a preparar su despliegue en un entorno externo con el objetivo de que pudiera estar disponible de forma continua y accesible desde Internet. Esta decisión responde a la necesidad de independizar la ejecución del sistema del equipo local de desarrollo, permitiendo que la aplicación pueda ser utilizada por usuarios reales, realizar pruebas con clientes de confianza y validar su comportamiento en un entorno similar al de producción.

Para ello se optó por contratar un servidor VPS, concretamente una instancia Linux alojada en Hetzner Cloud. La elección de un VPS frente a otras alternativas, como un hosting compartido, se debe a que la aplicación requiere un mayor control sobre el entorno de ejecución, instalación de dependencias específicas, uso de contenedores Docker, base de datos MySQL, almacenamiento persistente de documentos y ejecución de procesos relacionados con OCR. El servidor contratado proporciona los recursos necesarios para ejecutar la aplicación de forma estable, incluyendo CPU, memoria RAM y almacenamiento SSD suficiente para las primeras pruebas reales.

El sistema operativo seleccionado para el servidor fue Ubuntu, por tratarse de una distribución Linux estable, ampliamente utilizada en entornos de servidor y compatible con las herramientas necesarias para el despliegue. Una vez creado el servidor, se configuró el acceso mediante SSH, permitiendo la administración remota del VPS desde el equipo de desarrollo. Asimismo, se habilitó un firewall básico, tanto desde el proveedor como desde el propio sistema operativo,

permitiendo únicamente los puertos necesarios: SSH para administración, HTTP para acceso web inicial y HTTPS para el acceso seguro mediante certificado SSL.

Preparación del proyecto para producción

Antes de desplegar la aplicación en el servidor, fue necesario adaptar el proyecto para su ejecución en un entorno de producción. Para ello se configuraron perfiles diferenciados de Spring Boot, separando la configuración de desarrollo de la configuración de producción. El perfil de producción se definió mediante `application-prod.properties`, evitando incluir credenciales o datos sensibles directamente en el código fuente.

Las variables sensibles, como las credenciales de la base de datos, la contraseña del usuario administrador inicial, la ruta de almacenamiento de documentos, el tamaño máximo permitido para archivos y la configuración de cookies, se externalizaron mediante variables de entorno. Para gestionar esta configuración se utilizó un archivo `.env`, que se mantiene únicamente en el servidor y no se incluye en el repositorio. Además, se creó un archivo `.env.example` como plantilla de referencia, permitiendo documentar qué variables son necesarias sin exponer valores reales.

Esta separación entre código y configuración permite desplegar la misma aplicación en distintos entornos sin modificar el código fuente, simplemente cambiando las variables correspondientes. También mejora la seguridad del proyecto, ya que evita que contraseñas o datos privados queden versionados en GitHub.

Contenerización con Docker

Para facilitar el despliegue y garantizar que la aplicación se ejecuta siempre bajo las mismas condiciones, se utilizó Docker. Se creó un `Dockerfile` encargado de construir la imagen de la aplicación Spring Boot. Esta imagen incluye el entorno necesario para ejecutar el archivo `.jar` generado por Maven y las dependencias adicionales requeridas por la aplicación.

El arranque de la aplicación dentro del contenedor se realiza mediante el comando:

```
java -jar /app/app.jar
```

De esta forma, Docker se encarga de iniciar el contenedor y ejecutar la aplicación Spring Boot, que queda escuchando internamente en el puerto 8080.

Además del contenedor de la aplicación, se configuró un archivo `docker-compose.yml` para levantar todos los servicios necesarios de forma coordinada. La composición incluye, principalmente, el servicio de la aplicación y un servicio de base de datos MySQL. Esta estructura permite arrancar, detener, reconstruir y

mantener todo el entorno de forma sencilla mediante comandos de Docker Compose.

El uso de Docker Compose aporta varias ventajas importantes. En primer lugar, simplifica el despliegue, ya que todos los servicios se definen en un único archivo. En segundo lugar, facilita la reproducción del entorno tanto en local como en el servidor. Por último, permite aislar la aplicación y la base de datos, manteniendo una arquitectura más ordenada y fácil de mantener.

Configuración de la base de datos

La aplicación utiliza MySQL como sistema gestor de base de datos. En el entorno de producción, MySQL se ejecuta dentro de un contenedor Docker independiente. Se configuró una base de datos específica para la aplicación y un usuario propio con permisos limitados, evitando que la aplicación utilice directamente el usuario administrador root de MySQL.

La configuración de conexión a la base de datos se externalizó mediante variables de entorno, de forma que la URL, el usuario y la contraseña no quedan escritos directamente en el código. La aplicación, al arrancar con el perfil de producción, lee dichos valores desde el entorno del contenedor.

También se configuró la persistencia de la base de datos mediante volúmenes Docker. Esto garantiza que los datos no se pierdan al reiniciar o reconstruir los contenedores. Durante las pruebas iniciales, se verificó que la información almacenada permanecía disponible tras detener y volver a levantar el entorno, confirmando así que la persistencia estaba correctamente configurada.

Usuario administrador inicial

Uno de los aspectos importantes del despliegue fue la creación controlada del usuario administrador inicial. Durante el desarrollo existían usuarios de prueba, pero en producción no resulta adecuado mantener credenciales fijas o usuarios generados de forma automática sin control.

Por ello se implementó un mecanismo de inicialización que crea un usuario administrador únicamente cuando no existe ya ningún administrador en la base de datos. Las credenciales de este primer usuario se obtienen desde variables de entorno definidas en el archivo .env. De esta manera, el administrador inicial se puede crear en el primer arranque de la aplicación, pero no se vuelve a modificar en reinicios posteriores si ya existe.

Este comportamiento es importante porque evita que la contraseña del administrador se restablezca accidentalmente cada vez que se reinicia la

aplicación. También permite que las credenciales reales del entorno de producción se mantengan fuera del repositorio.

Almacenamiento persistente de documentos

La aplicación permite subir, visualizar, descargar, editar y procesar documentos PDF. Por este motivo, fue necesario configurar un sistema de almacenamiento persistente para los archivos subidos por los usuarios.

En el entorno Docker, la aplicación guarda los documentos dentro del contenedor en la ruta configurada mediante la variable `UPLOAD_DIR`. Esta ruta se monta sobre una carpeta real del servidor mediante un volumen, de forma que los archivos no quedan almacenados únicamente dentro del contenedor, sino en el sistema de archivos del VPS.

La estructura configurada permite que, aunque el contenedor de la aplicación se reconstruya o se reinicie, los documentos permanezcan disponibles. La base de datos almacena la información necesaria para identificar cada documento, como su nombre original, nombre interno, tipo de documento, usuario que lo subió y entidad asociada, mientras que el archivo físico se guarda en la carpeta de uploads del servidor.

Esta separación entre metadatos en base de datos y archivo físico en disco es fundamental para el correcto funcionamiento del sistema documental. Además, se tuvo en cuenta la necesidad de incluir esta carpeta en las copias de seguridad, ya que la base de datos por sí sola no contiene el contenido de los documentos.

Configuración del OCR en producción

Una de las funcionalidades más complejas del sistema es el procesamiento OCR de documentos PDF. Esta funcionalidad permite analizar documentos escaneados, extraer texto de sus páginas y detectar automáticamente diferentes tipos documentales. Para ello se utiliza Tesseract OCR.

Durante el despliegue fue necesario adaptar esta funcionalidad al entorno Linux del contenedor Docker. En primer lugar, se instalaron las dependencias necesarias dentro de la imagen de la aplicación, incluyendo Tesseract y los paquetes de idioma correspondientes, especialmente español. También se configuró correctamente la ruta de los archivos tessdata, que contienen los modelos de idioma utilizados por Tesseract.

Inicialmente se planteó el uso de Tess4J como librería Java para la integración directa con Tesseract. Sin embargo, durante las pruebas en el entorno Linux se detectaron problemas relacionados con librerías nativas de Leptonica. Para mejorar la estabilidad del despliegue, se optó por ejecutar Tesseract como proceso del sistema desde la aplicación Java. De esta forma, la aplicación sigue

controlando el flujo completo del OCR, pero delega la lectura del texto al ejecutable de Tesseract instalado dentro del contenedor.

El flujo final consiste en convertir cada página del PDF en una imagen temporal, ejecutar Tesseract sobre dicha imagen, recoger el texto resultante y analizarlo para determinar el tipo de documento. Esta solución permite aprovechar el motor OCR instalado en el contenedor, evitando problemas derivados de incompatibilidades entre librerías nativas y manteniendo la funcionalidad operativa en producción.

Configuración de Nginx como proxy inverso

Una vez comprobado que la aplicación funcionaba correctamente dentro del VPS, se configuró Nginx como proxy inverso. La aplicación Spring Boot se ejecuta internamente en el puerto 8080, pero dicho puerto no se expone directamente al exterior. En su lugar, Nginx recibe las peticiones web en los puertos estándar HTTP y HTTPS y las redirige internamente hacia la aplicación.

Esta configuración aporta varias ventajas. En primer lugar, permite acceder a la aplicación sin indicar manualmente el puerto 8080. En segundo lugar, mejora la seguridad, ya que el puerto interno de la aplicación queda limitado al propio servidor. En tercer lugar, permite integrar certificados SSL y gestionar el acceso HTTPS de forma más sencilla.

La configuración de Nginx incluye también el ajuste del tamaño máximo de subida de archivos, necesario para permitir la carga de documentos PDF de mayor tamaño. Además, se configuraron las cabeceras necesarias para que Spring Boot reciba correctamente la información original de la petición, como el host, la IP del cliente y el protocolo utilizado.

Acceso mediante dominio y HTTPS

Tras comprobar que la aplicación funcionaba correctamente mediante la IP pública del servidor, se preparó el acceso mediante dominio. Para ello fue necesario solicitar al administrador del dominio la creación de un registro DNS de tipo A, apuntando el subdominio elegido hacia la IP pública del VPS.

Una vez propagada la DNS, se configuró Nginx para responder al nombre de dominio correspondiente. Posteriormente, se preparó la aplicación para funcionar bajo HTTPS mediante Certbot y certificados SSL. El uso de HTTPS es fundamental en una aplicación que gestiona usuarios, contraseñas y documentos, ya que garantiza que la comunicación entre el navegador y el servidor se realiza de forma cifrada.

Seguridad básica del servidor

Durante la puesta en marcha del VPS se aplicaron varias medidas básicas de seguridad. Se configuró el acceso remoto mediante SSH, se activó el firewall del servidor y se limitaron los puertos abiertos al exterior. Únicamente quedaron expuestos los puertos necesarios para administración y acceso web.

No se expuso públicamente el puerto de MySQL, ya que la base de datos solo debe ser accesible desde los contenedores internos. Tampoco se expuso directamente el puerto 8080 de Spring Boot, quedando la aplicación protegida detrás de Nginx.

Asimismo, se mantuvo fuera del repositorio cualquier archivo con credenciales reales, especialmente el archivo `.env`, y se estableció una plantilla segura `.env.example` para documentar las variables necesarias. Esta práctica evita fugas accidentales de información sensible en el control de versiones.

Comprobaciones realizadas

Durante el proceso de despliegue se realizaron varias pruebas para validar que el sistema funcionaba correctamente en producción. Entre las comprobaciones realizadas destacan:

- Arranque correcto de los contenedores Docker.
- Conexión de la aplicación con MySQL.
- Creación del usuario administrador inicial.
- Acceso a la aplicación desde el navegador.
- Inicio de sesión como administrador.
- Creación de clientes y usuarios cliente.
- Creación y consulta de solicitudes.
- Subida de documentos PDF.
- Visualización, descarga y edición de documentos.
- Persistencia de documentos tras reiniciar contenedores.
- Persistencia de datos en MySQL.
- Funcionamiento del OCR en el entorno del servidor.
- Acceso a la aplicación a través de Nginx.
- Preparación del acceso mediante dominio y HTTPS.

Estas pruebas permitieron validar que la aplicación podía funcionar fuera del entorno local de desarrollo y que estaba preparada para iniciar una fase de pruebas con usuarios reales.

Mantenimiento y actualización

Una vez desplegada la aplicación, se definió un flujo de actualización basado en Git y Docker. El desarrollo continúa realizándose en local, donde se prueban los

cambios antes de subirlos al repositorio. Cuando una versión está preparada, se realiza un commit y un push a GitHub. Posteriormente, en el servidor se actualiza el código mediante git pull y se reconstruye la imagen de la aplicación con Docker Compose.

El comando habitual de actualización en el servidor es:

docker compose up -d --build

Este comando reconstruye la imagen de la aplicación y levanta el contenedor actualizado sin eliminar los datos persistentes. Se estableció también la precaución de no utilizar docker compose down -v en producción, ya que este comando puede eliminar volúmenes de Docker y provocar la pérdida de datos de la base de datos.

Además, se definió la necesidad de realizar copias de seguridad periódicas tanto de la base de datos como de la carpeta de documentos. La base de datos contiene la información estructurada del sistema, mientras que la carpeta de uploads contiene los archivos físicos subidos por los usuarios. Ambos elementos son necesarios para poder restaurar correctamente la aplicación en caso de fallo.

Conclusión del despliegue

El despliegue de la aplicación en un servidor VPS ha permitido transformar el proyecto desde un entorno exclusivamente local a un sistema accesible desde Internet y preparado para pruebas reales. La combinación de Spring Boot, MySQL, Docker, Nginx y Tesseract OCR proporciona una arquitectura flexible y escalable, adecuada para una aplicación de gestión documental.

El uso de Docker facilita la reproducción del entorno, simplifica el despliegue y mejora la mantenibilidad. La configuración mediante variables de entorno permite separar el código de los datos sensibles, aumentando la seguridad. La utilización de Nginx como proxy inverso permite exponer la aplicación de forma ordenada y preparar el acceso seguro mediante HTTPS. Finalmente, la integración del OCR en el entorno del servidor permite mantener operativas las funcionalidades avanzadas de tratamiento documental.

Con este proceso, la aplicación queda preparada para iniciar una fase de pruebas con clientes de confianza, validar su comportamiento en un entorno real y continuar incorporando mejoras futuras sobre una base de despliegue estable.

Resultado de la implementación

Como resultado de la implementación se ha obtenido una aplicación web funcional para la gestión documental de trámites administrativos relacionados con vehículos. El sistema permite diferenciar el acceso entre usuarios administradores y usuarios clientes, ofreciendo a cada perfil las funcionalidades correspondientes.

Los usuarios cliente pueden iniciar sesión, acceder a su panel, crear solicitudes de trámite, consultar el estado de sus solicitudes y expedientes, y aportar documentación asociada. Los usuarios administradores pueden consultar las solicitudes recibidas, convertir solicitudes en expedientes, crear expedientes manualmente, revisar la documentación aportada, gestionar interesados, registrar incidencias, consultar el historial, revisar mensajes y realizar el seguimiento general de los trámites.

La aplicación centraliza la información de clientes, usuarios, solicitudes, expedientes, documentos, interesados, incidencias, mensajes e historial en un único sistema. Esto evita la dispersión de datos, mejora el control interno y facilita la comunicación entre la gestoría y sus clientes.

Además, la incorporación del procesamiento OCR y la división automática de expedientes completos aporta una funcionalidad avanzada que permite reducir tareas manuales y mejorar la organización documental.

En conjunto, la implementación cumple el objetivo principal del proyecto: proporcionar una herramienta web que facilite la gestión documental, mejore el seguimiento de los trámites y permita a la gestoría trabajar de forma más ordenada, centralizada y eficiente.

Gracias al despliegue, la aplicación es accesible a través de la URL

<https://app.gestoriacn.com>

Se ha proporcionado un usuario a todos los empleados de la gestoría y a algunos clientes de confianza que nos han solicitado acceso para realizar las pruebas.

Se ha habilitado un usuario de pruebas con rol cliente para la evaluación de la aplicación. Las credenciales serán facilitadas al tribunal/profesorado por un canal independiente.

METODOLOGÍA DE DESARROLLO

Enfoque metodológico

Para el desarrollo del proyecto se ha seguido una metodología incremental. Este enfoque consiste en construir la aplicación de forma progresiva, dividiendo el trabajo en fases y módulos funcionales que se van completando, probando y mejorando a medida que avanza el desarrollo.

Esta metodología resulta adecuada para el proyecto porque la aplicación está formada por diferentes partes relacionadas entre sí. En lugar de intentar desarrollar todo el sistema completo desde el inicio, se fue construyendo por bloques: primero la base del proyecto, después la autenticación, posteriormente la gestión de solicitudes y expedientes, la gestión documental, las incidencias, los interesados y, finalmente, funcionalidades más avanzadas como el OCR, el historial y la mensajería interna.

El enfoque incremental permitió validar cada parte antes de continuar con la siguiente. De esta forma, los errores podían detectarse antes y los cambios podían incorporarse de manera más controlada.

Planificación inicial del proyecto

Antes de comenzar la implementación se realizó una planificación inicial del proyecto. Esta planificación tenía como objetivo organizar el trabajo, estimar el tiempo necesario para cada fase y establecer un orden lógico de desarrollo.

La planificación inicial del proyecto se representó mediante un diagrama de Gantt, en el que se distribuyeron temporalmente las principales fases del desarrollo: análisis de requisitos, diseño del sistema, diseño de la base de datos, implementación, pruebas y documentación. Debido a su formato horizontal y a la necesidad de mantener una correcta legibilidad, el diagrama completo se incluye al final de la memoria como **Anexo I**.

Fase	Horas estimadas
Análisis de requisitos	6
Diseño del sistema	8
Diseño de base de datos	6
Implementación backend	15
Desarrollo interfaz	8
Pruebas	4
Documentación	6

Total estimado: 53 horas

Esta planificación sirvió como guía general durante el desarrollo, aunque no se mantuvo de forma completamente rígida. A medida que el proyecto avanzó, algunas tareas requirieron más tiempo del previsto y fue necesario incorporar nuevas funcionalidades que no estaban contempladas inicialmente.

Fases del proyecto

El desarrollo comenzó con una fase de análisis, en la que se identificaron las necesidades que debía cubrir la aplicación. Se estudiaron los procesos habituales de una gestoría dedicada a trámites de vehículos y se definieron las funcionalidades principales que debía ofrecer el sistema. Durante esta etapa me entrevisté con varios clientes y empleados para determinar qué funciones debía incorporar la aplicación y cuáles eran sus necesidades reales, esto aportó una gran variedad de ideas.

Después se realizó el diseño inicial, incluyendo los casos de uso, el modelo entidad-relación, el diagrama UML, el modelo de base de datos y los mockups de la interfaz. Esta fase permitió establecer una primera visión del sistema antes de comenzar la programación. Este paso fue fundamental porque es la base de la aplicación, se prestó especial atención en que todas las relaciones fueran correctas, para que la información fuera segura, bien indexada y sin brechas de seguridad en los accesos.

A continuación, se inició la implementación técnica. Primero se creó la estructura del proyecto Spring Boot y se configuraron las dependencias principales. Después se desarrollaron las entidades, repositorios, servicios y controladores necesarios para construir las funcionalidades básicas.

Una vez implementada la base del sistema, se añadieron las funcionalidades principales de gestión de solicitudes, expedientes, documentos e incidencias. Posteriormente se incorporaron mejoras como la gestión de interesados, la mensajería interna, el historial

de actividad y el procesamiento OCR de documentos. Al mismo tiempo que se generaban los diversos controladores y sus endpoints, se fueron generando las vistas con sus respectivas plantillas, apoyadas siempre en un layout principal.

Durante el desarrollo se realizaron diversas reuniones con los clientes y empleados que habían participado en las reuniones previas, con el objetivo de recoger feedback sobre los resultados obtenidos, tanto a nivel visual como en relación con la usabilidad de la interfaz.

Finalmente, se realizaron pruebas funcionales y ajustes visuales, corrigiendo errores detectados durante el uso de la aplicación y mejorando la organización de las pantallas.

Pruebas del sistema

Una vez desarrolladas las funcionalidades principales de la aplicación se realizaron pruebas para verificar el correcto funcionamiento del sistema.

Estas pruebas permitieron comprobar que las diferentes funcionalidades de la aplicación cumplen los requisitos definidos en la fase inicial del proyecto.

Entre los aspectos verificados se encuentran la autenticación de usuarios, la creación de solicitudes, la gestión de expedientes, la subida de documentación y el registro de incidencias.

Una de las pruebas más importantes era la seguridad, haciendo login con diversos usuarios de varios clientes y también del usuario admin. Se comprobó que cada usuario solo podía acceder a sus propios expedientes y que no podía hacer las acciones para las que no estuviera autorizado.

También se realizaron numerosas pruebas sobre el apartado de OCR, subiendo varios archivos de ejemplo y ajustando los patrones de identificación para mejorar la catalogación automática de documentos, obteniendo resultados bastante positivos.

Desviaciones respecto a la planificación inicial

Durante el desarrollo se produjeron algunas desviaciones respecto a la planificación inicial. Estas desviaciones estuvieron relacionadas principalmente con la ampliación del alcance funcional del proyecto.

La incorporación del OCR, la división automática de PDFs, el historial de actividad, la mensajería interna y la mejora visual de la interfaz hicieron que algunas fases requirieran más tiempo del previsto. Especialmente la gestión documental y el tratamiento automático de archivos PDF supusieron un mayor esfuerzo técnico.

A pesar de ello, estas desviaciones se consideran justificadas, ya que permitieron obtener una aplicación más completa y útil. La planificación inicial sirvió como guía, pero la metodología incremental permitió adaptar el proyecto a medida que surgían nuevas necesidades.

Como conclusión, el hecho de que la aplicación suponga un uso real y que se va a implantar de manera profesional directamente con nuestros clientes ha supuesto una motivación adicional a realizarla de la forma más completa posible, aunque he tratado de acotar al máximo posible el alcance del proyecto a una complejidad razonable.

Valoración de la metodología utilizada

La metodología incremental utilizada durante el desarrollo ha sido adecuada para este proyecto. Al dividir el trabajo en fases y módulos funcionales, fue posible avanzar de forma ordenada, validar cada parte del sistema y realizar mejoras progresivas.

Este enfoque permitió que el proyecto evolucionara desde una idea inicial más sencilla hasta una aplicación más completa, incorporando nuevas entidades, nuevas funcionalidades y mejoras visuales sin perder la estructura general del sistema.

Además, la metodología seguida facilitó la detección de errores, la adaptación del modelo de datos y la mejora de la interfaz conforme se iba construyendo la aplicación.

En conclusión, el uso de una metodología incremental permitió desarrollar una aplicación funcional, organizada y preparada para futuras ampliaciones, manteniendo una relación coherente entre el análisis inicial, el diseño, la implementación y el resultado final.

README y GIT

Para la organización y seguimiento del proyecto se ha utilizado Git como sistema de control de versiones, junto con GitHub como repositorio remoto. Esta decisión ha permitido mantener un historial de los cambios realizados durante el desarrollo y conservar una copia del proyecto accesible desde distintos equipos.

El uso de Git ha sido importante porque el proyecto ha ido evolucionando de forma progresiva. A lo largo del desarrollo se han incorporado nuevas funcionalidades, como la gestión de solicitudes, expedientes, documentos, interesados, incidencias, historial, mensajes y procesamiento OCR. Gracias al control de versiones, cada una de estas fases ha podido registrarse mediante commits, facilitando el seguimiento de los avances y la posibilidad de volver a versiones anteriores si fuera necesario.

GitHub se ha utilizado como repositorio remoto para almacenar el código fuente de la aplicación. Esto ha permitido trabajar de forma más segura y evitar depender únicamente de una copia local del proyecto. Además, facilita el desarrollo desde distintos equipos, ya que los cambios pueden sincronizarse mediante operaciones como commit, push y pull.

Junto al repositorio se incluye un archivo README, que sirve como documentación básica del proyecto. Este archivo recoge la información necesaria para comprender la finalidad de la aplicación, las tecnologías empleadas y los pasos principales para su ejecución.

El README contiene una descripción general del sistema de gestión documental, indicando que se trata de una aplicación web desarrollada con Spring Boot para gestionar solicitudes, expedientes, documentos e incidencias en una gestoría de trámites relacionados con vehículos. También incluye las tecnologías utilizadas, como Java, Spring Boot, Spring Security, Spring Data JPA, Thymeleaf, Bootstrap, MySQL, Maven, Apache PDFBox, Tess4J y Tesseract OCR.

Además, el README puede incluir los requisitos necesarios para ejecutar el proyecto, como disponer de Java, Maven, MySQL y Tesseract OCR instalados. También se pueden indicar los pasos básicos de configuración, como crear la base de datos, revisar el archivo de configuración de la aplicación y ejecutar el proyecto desde el entorno de desarrollo o mediante Maven.

Es importante que el README no incluya información sensible, como contraseñas reales, claves privadas o datos personales. En caso de necesitar credenciales de prueba, estas deben ser usuarios ficticios creados únicamente para la demostración del proyecto.

En conclusión, el uso de Git, GitHub y el archivo README permite presentar el proyecto de una forma más organizada, profesional y mantenible. Git facilita el control de cambios, GitHub actúa como copia remota del código y el README proporciona una guía inicial para comprender y ejecutar la aplicación.

Enlace a repositorio GitHub:

El código fuente del proyecto se encuentra alojado en un repositorio remoto de GitHub, utilizado como sistema de respaldo, control de versiones y consulta del código desarrollado.

El repositorio permite revisar la estructura completa de la aplicación, incluyendo las entidades, repositorios, servicios, controladores, vistas, archivos de configuración, recursos estáticos y documentación básica del proyecto.

Enlace al repositorio:

https://github.com/SamNegDev/gestor_documental

TRABAJO FUTURO

Aunque la aplicación desarrollada cumple con los objetivos principales planteados inicialmente, el sistema presenta un amplio margen de ampliación. Al haberse construido sobre una arquitectura en capas, con una base de datos relacional normalizada y una separación clara entre entidades, servicios, controladores y vistas, la aplicación puede evolucionar hacia nuevas funcionalidades sin necesidad de modificar completamente su estructura principal.

Una de las posibles líneas de mejora sería la **automatización de la recepción y registro de documentación mediante canales externos**, como correo electrónico o WhatsApp. Actualmente, la documentación se incorpora a la plataforma mediante subida manual por parte del usuario. Sin embargo, en un entorno real de gestión es habitual que los clientes envíen documentos por email o mensajería instantánea. Una futura ampliación podría permitir que el sistema recibiera automáticamente estos archivos, los asociara al cliente correspondiente y los registrara dentro de la solicitud o expediente adecuado. Esta mejora reduciría el trabajo manual y permitiría centralizar documentación que actualmente puede quedar dispersa en distintos canales.

Otra ampliación interesante sería la **integración de avisos automáticos por correo electrónico o WhatsApp**. Esta funcionalidad permitiría notificar al cliente cuando se genera una incidencia, cuando falta documentación, cuando un expediente cambia de estado o cuando se completa un trámite. De esta forma, la comunicación con el cliente sería más rápida y se evitaría que el administrador tuviera que avisar manualmente en cada caso. Además, este sistema de notificaciones podría mejorar la experiencia del cliente, ya que recibiría información actualizada sobre el estado de sus trámites.

También sería muy útil incorporar un módulo de **control de indicadores clave del negocio**, conocidos como KPIs. Este módulo permitiría analizar datos como el tiempo medio de tramitación de los expedientes, el tiempo de respuesta a solicitudes, el número de incidencias generadas, los expedientes finalizados por periodo o la carga de trabajo acumulada. Estos indicadores ayudarían a la gestión a tomar mejores decisiones, detectar cuellos de botella y medir la eficiencia de sus procesos internos.

Dentro de esta misma línea, se podría desarrollar un apartado específico para **profesionales o colaboradores**, especialmente orientado a empresas o clientes recurrentes. En este apartado podrían consultar información sobre sus operaciones, ventas por periodos, precios aplicados, trámites realizados, expedientes pendientes o documentación aportada. Esta funcionalidad

convertiría la aplicación no solo en una herramienta documental, sino también en una plataforma de seguimiento comercial y operativo.

Otra mejora futura sería la incorporación de un sistema de **cálculo de presupuestos**. La aplicación podría permitir generar presupuestos en función del tipo de trámite, el cliente, los servicios incluidos, tasas, honorarios y otros conceptos configurables. Esta funcionalidad sería especialmente útil para automatizar parte del trabajo administrativo previo al inicio del expediente y permitiría mantener un registro histórico de presupuestos aceptados, rechazados o pendientes de respuesta.

Una de las ampliaciones más interesantes sería la **lectura avanzada de documentos y generación automática de archivos XML** con los datos necesarios para su posterior volcado en aplicaciones externas de gestión. Actualmente, el sistema ya incorpora procesamiento OCR y clasificación documental, lo que supone una base muy importante para avanzar hacia esta funcionalidad. En una versión futura, la aplicación podría extraer datos concretos de documentos como DNI, permisos de circulación, fichas técnicas, contratos o facturas, estructurarlos y generar un archivo XML compatible con otros programas utilizados por la gestoría. Esta mejora permitiría reducir todavía más la introducción manual de datos y aumentaría considerablemente el nivel de automatización del sistema.

Además, el sistema podría ampliarse para incorporar una gestión más completa de usuarios, permisos y perfiles. Aunque actualmente se diferencian los roles de cliente y administrador, en el futuro podrían añadirse perfiles más específicos, como gestor, supervisor, comercial o profesional externo. Cada perfil podría tener permisos concretos sobre determinadas funcionalidades, expedientes o clientes.

También sería posible mejorar la parte de trazabilidad mediante un historial más avanzado, permitiendo registrar con mayor detalle cada acción realizada sobre un expediente: cambios de estado, documentos añadidos, incidencias resueltas, mensajes enviados o modificaciones realizadas por cada usuario. Esta información podría presentarse como una línea temporal del expediente, facilitando la consulta de todo el proceso de tramitación.

Otra posible mejora sería la creación de un panel de administración más avanzado, con gráficos, estadísticas y resúmenes visuales. Este panel podría mostrar solicitudes pendientes, expedientes en curso, incidencias abiertas, documentos pendientes de revisión, tiempos medios de gestión y evolución del volumen de trabajo. De esta forma, el administrador tendría una visión global del estado del negocio desde una única pantalla.

Todas estas posibles ampliaciones demuestran que la aplicación desarrollada es escalable y puede evolucionar hacia un sistema más completo. El proyecto no se

limita a una herramienta básica de almacenamiento documental, sino que establece una base sólida sobre la que pueden construirse nuevas funcionalidades relacionadas con automatización, comunicación, análisis de negocio, integración con sistemas externos y mejora de procesos internos.

En definitiva, el sistema desarrollado constituye una primera versión funcional, pero preparada para crecer. Su estructura permite adaptarlo a nuevas necesidades de la gestoría e incluso extenderlo a otros servicios administrativos similares, manteniendo como objetivo principal la centralización de la información, la reducción del trabajo manual y la mejora del seguimiento de los trámites.

CONCLUSIONES

El desarrollo de este proyecto ha permitido construir una aplicación web orientada a la gestión documental de trámites administrativos relacionados con vehículos. La finalidad principal era crear una herramienta que facilitara la comunicación entre la gestoría y sus clientes, centralizara la documentación y permitiera realizar un seguimiento más ordenado de solicitudes, expedientes, documentos e incidencias.

A lo largo del proyecto se ha conseguido implementar un sistema funcional con dos perfiles principales de usuario: cliente y administrador. El usuario cliente puede acceder a su área privada, crear solicitudes, consultar expedientes y aportar documentación. Por su parte, el administrador puede revisar solicitudes, crear expedientes, gestionar documentos, registrar incidencias, asociar interesados, consultar mensajes y realizar el seguimiento completo de los trámites. Esta separación de roles permite adaptar la aplicación a las necesidades de cada tipo de usuario.

Uno de los aspectos más importantes del proyecto ha sido la evolución del modelo inicial. Durante el desarrollo se detectó que algunas funcionalidades requerían ampliar el diseño previsto inicialmente. Por este motivo, se incorporaron entidades como Historial y Mensaje, además de otras entidades auxiliares y de relación que permiten representar mejor la lógica real del sistema. Esta evolución no se considera una desviación negativa, sino una mejora del planteamiento inicial que ha permitido obtener una aplicación más completa y cercana a un escenario real.

La gestión documental ha sido uno de los elementos centrales del sistema. La aplicación no solo permite subir y consultar archivos, sino que también almacena sus metadatos, los asocia a solicitudes o expedientes y controla los permisos de acceso. Además, la incorporación del procesamiento OCR y la división automática de documentos PDF supone un valor añadido importante, ya que permite avanzar hacia una gestión más automatizada de la documentación. Esta funcionalidad diferencia el proyecto de una aplicación básica de almacenamiento de archivos y lo acerca a una herramienta más especializada.

Desde el punto de vista técnico, el uso de Java, Spring Boot, Spring Security, Spring Data JPA, Thymeleaf, MySQL y Bootstrap ha permitido desarrollar una aplicación estructurada, mantenible y preparada para futuras ampliaciones. La arquitectura en capas ha facilitado la separación de responsabilidades entre controladores, servicios, repositorios y entidades, lo que mejora la organización del código y facilita su comprensión.

El proyecto también ha permitido aplicar conocimientos relacionados con el diseño de bases de datos, la seguridad web, la gestión de formularios, la persistencia de datos, el control de roles, la subida de archivos, el tratamiento de documentos PDF y la integración de OCR.

En cuanto a la interfaz, el diseño final ha evolucionado respecto a los mockups iniciales. Durante la implementación fue necesario adaptar las pantallas a las funcionalidades reales del sistema, reorganizar la información y mejorar la usabilidad. Esto permitió obtener una interfaz más completa, clara y ajustada al funcionamiento real de la aplicación.

La metodología incremental utilizada ha resultado adecuada, ya que permitió desarrollar el proyecto por fases, validar cada módulo de forma progresiva y adaptar el diseño a las necesidades que fueron apareciendo. Gracias a este enfoque, fue posible incorporar mejoras importantes sin perder la estructura general del sistema.

Como valoración final, se puede concluir que los objetivos principales del proyecto se han cumplido. Se ha desarrollado una aplicación web capaz de gestionar solicitudes, expedientes, documentos, interesados, incidencias, mensajes e historial dentro de un entorno centralizado. Además, se ha incorporado una funcionalidad avanzada de tratamiento documental mediante OCR, que aporta mayor valor al sistema y abre la puerta a futuras automatizaciones.

El resultado final es una aplicación funcional, escalable y orientada a resolver una necesidad real dentro del ámbito de una gestoría. Aunque todavía existen muchas posibilidades de mejora, el proyecto establece una base sólida sobre la que seguir evolucionando hacia una plataforma más completa de gestión, comunicación y automatización documental.

A nivel personal, este proyecto ha supuesto una oportunidad para aplicar de forma conjunta muchos de los conocimientos adquiridos durante el ciclo formativo. El desarrollo de una aplicación completa, desde el análisis inicial hasta la implementación final, me ha permitido comprender mejor la importancia de la planificación, el diseño de la base de datos, la organización del código, la seguridad y la experiencia de usuario. Además, trabajar sobre una necesidad real de mi propia actividad profesional ha supuesto una motivación adicional, ya que he podido aplicar lo aprendido a la mejora de los procesos internos de mi negocio. Esta fue, sin duda, una de las principales razones por las que decidí iniciar este ciclo formativo. El resultado final me produce una gran satisfacción, tanto por el trabajo realizado durante estos dos años como por las posibilidades que abre de cara al futuro, permitiendo seguir desarrollando nuevos proyectos y llevar la gestoría a un nivel superior de eficiencia y diferenciación frente a la competencia.

REFERENCIAS

Apache Maven Project. (s. f.). *Maven documentation*. Apache Maven.

Recuperado el 26 de abril de 2026, de <https://maven.apache.org/guides/>

Apache PDFBox. (s. f.). *Apache PDFBox: A Java PDF library*. The Apache Software Foundation. Recuperado el 26 de abril de 2026, de <https://pdfbox.apache.org/>

Apache Tomcat. (s. f.). *Apache Tomcat 10 documentation index*. The Apache Software Foundation. Recuperado el 26 de abril de 2026, de <https://tomcat.apache.org/tomcat-10.1-doc/index.html>

Bootstrap. (s. f.). *Get started with Bootstrap*. Recuperado el 26 de abril de 2026, de <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

Chart.js. (s. f.). *Getting started*. Recuperado el 26 de abril de 2026, de <https://www.chartjs.org/docs/latest/getting-started/>

Git. (s. f.). *Git documentation*. Recuperado el 26 de abril de 2026, de <https://git-scm.com/docs>

GitHub. (s. f.). *Repositories documentation*. GitHub Docs. Recuperado el 26 de abril de 2026, de <https://docs.github.com/en/repositories>

Hibernate. (s. f.). *Hibernate ORM user guide*. Recuperado el 26 de abril de 2026, de https://docs.hibernate.org/stable/orm/userguide/html_single/

JetBrains. (s. f.). *Getting started*. IntelliJ IDEA Documentation. Recuperado el 26 de abril de 2026, de <https://www.jetbrains.com/help/idea/getting-started.html>

MDN Web Docs. (s. f.). *CSS: Cascading Style Sheets*. Mozilla. Recuperado el 26 de abril de 2026, de <https://developer.mozilla.org/en-US/docs/Web/CSS>

MDN Web Docs. (s. f.). *HTML: HyperText Markup Language*. Mozilla. Recuperado el 26 de abril de 2026, de <https://developer.mozilla.org/en-US/docs/Web/HTML>

MDN Web Docs. (s. f.). *JavaScript*. Mozilla. Recuperado el 26 de abril de 2026, de <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

Oracle. (s. f.). *Java Platform, Standard Edition documentation*. Oracle Help Center. Recuperado el 26 de abril de 2026, de <https://docs.oracle.com/en/java/javase/>

Oracle. (s. f.). *MySQL 8.4 reference manual*. MySQL Documentation. Recuperado el 26 de abril de 2026, de <https://dev.mysql.com/doc/en/>

Oracle. (s. f.). *MySQL Connector/J developer guide*. MySQL Documentation. Recuperado el 26 de abril de 2026, de <https://dev.mysql.com/doc/connector-j/en/>

Oracle. (s. f.). *MySQL Workbench reference manual*. MySQL Documentation.

Recuperado el 26 de abril de 2026, de

<https://dev.mysql.com/doc/workbench/en/>

Project Lombok. (s. f.). *Project Lombok features*. Recuperado el 26 de abril de

2026, de <https://projectlombok.org/features/>

Spring. (s. f.). *Spring Boot reference documentation*. Spring Documentation.

Recuperado el 26 de abril de 2026, de <https://docs.spring.io/spring-boot/reference/index.html>

Spring. (s. f.). *Spring Data JPA*. Spring Documentation. Recuperado el 26 de abril

de 2026, de <https://docs.spring.io/spring-data/jpa/reference/index.html>

Spring. (s. f.). *Spring Framework documentation*. Spring Documentation.

Recuperado el 26 de abril de 2026, de <https://docs.spring.io/spring-framework/reference/index.html>

Spring. (s. f.). *Spring Security reference*. Spring Documentation. Recuperado el 26

de abril de 2026, de <https://docs.spring.io/spring-security/reference/index.html>

Tesseract OCR. (s. f.). *Tesseract documentation*. Recuperado el 26 de abril de

2026, de <https://tesseract-ocr.github.io/>

Tess4J. (s. f.). *Tess4J: JNA wrapper for Tesseract OCR API*. SourceForge.

Recuperado el 26 de abril de 2026, de <https://tess4j.sourceforge.net/>

Thymeleaf. (s. f.). *Thymeleaf documentation*. Recuperado el 26 de abril de 2026,

de <https://www.thymeleaf.org/documentation.html>

Ultraq. (s. f.). *Thymeleaf Layout Dialect*. Recuperado el 26 de abril de 2026, de

<https://ultraq.github.io/thymeleaf-layout-dialect/>

ANEXO 1

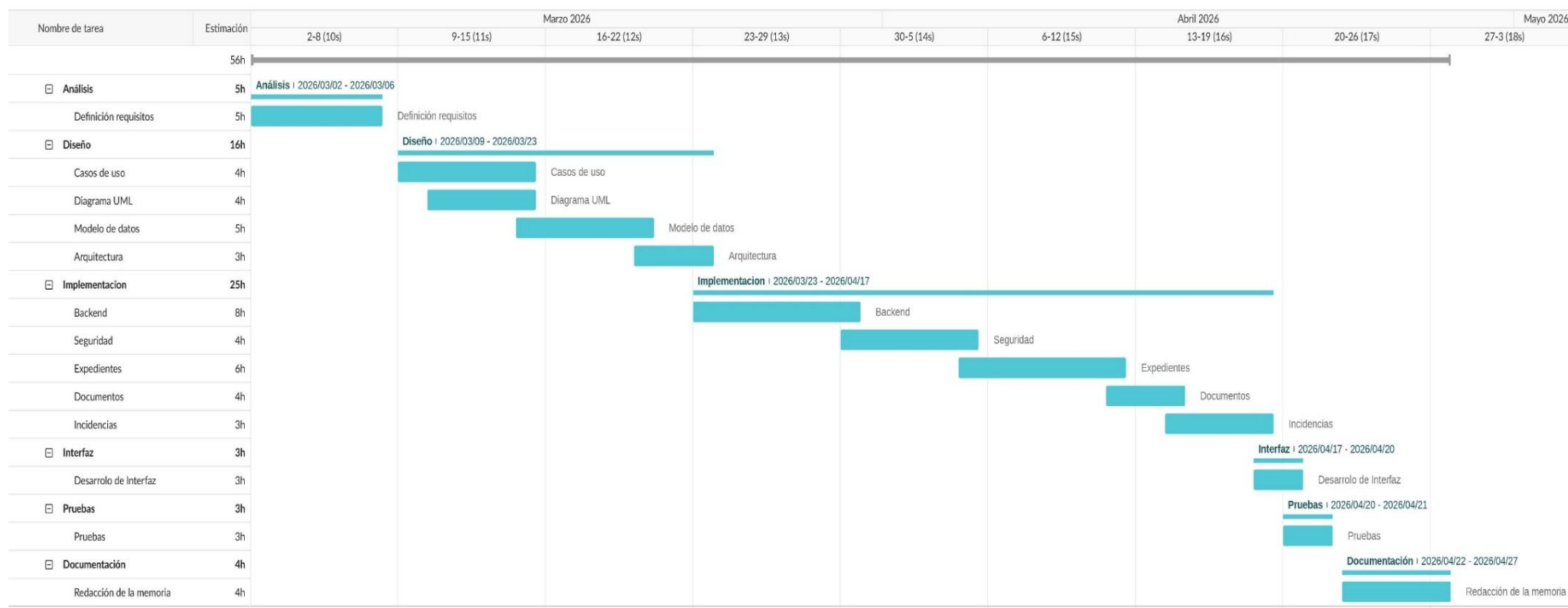


Ilustración 26. Diagrama de Gantt de la planificación del proyecto